

Which attribute is the best classifier?

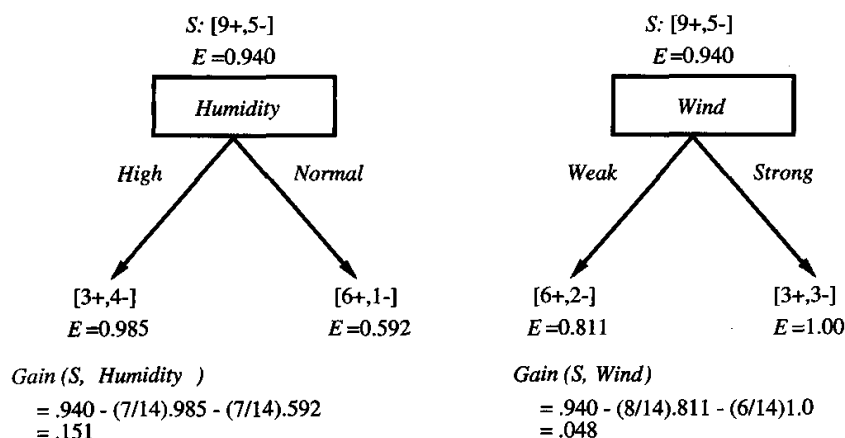


FIGURE 3.3

Humidity provides greater information gain than *Wind*, relative to the target classification. Here, E stands for entropy and S for the original collection of examples. Given an initial collection S of 9 positive and 5 negative examples, $[9+, 5-]$, sorting these by their *Humidity* produces collections of $[3+, 4-]$ (*Humidity* = High) and $[6+, 1-]$ (*Humidity* = Normal). The information gained by this partitioning is .151, compared to a gain of only .048 for the attribute *Wind*.

3.4.2 An Illustrative Example

To illustrate the operation of ID3, consider the learning task represented by the training examples of Table 3.2. Here the target attribute *PlayTennis*, which can have values *yes* or *no* for different Saturday mornings, is to be predicted based on other attributes of the morning in question. Consider the first step through

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

TABLE 3.2

Training examples for the target concept *PlayTennis*.

the algorithm, in which the topmost node of the decision tree is created. Which attribute should be tested first in the tree? ID3 determines the information gain for each candidate attribute (i.e., *Outlook*, *Temperature*, *Humidity*, and *Wind*), then selects the one with highest information gain. The computation of information gain for two of these attributes is shown in Figure 3.3. The information gain values for all four attributes are

$$\text{Gain}(S, \text{Outlook}) = 0.246$$

$$\text{Gain}(S, \text{Humidity}) = 0.151$$

$$\text{Gain}(S, \text{Wind}) = 0.048$$

$$\text{Gain}(S, \text{Temperature}) = 0.029$$

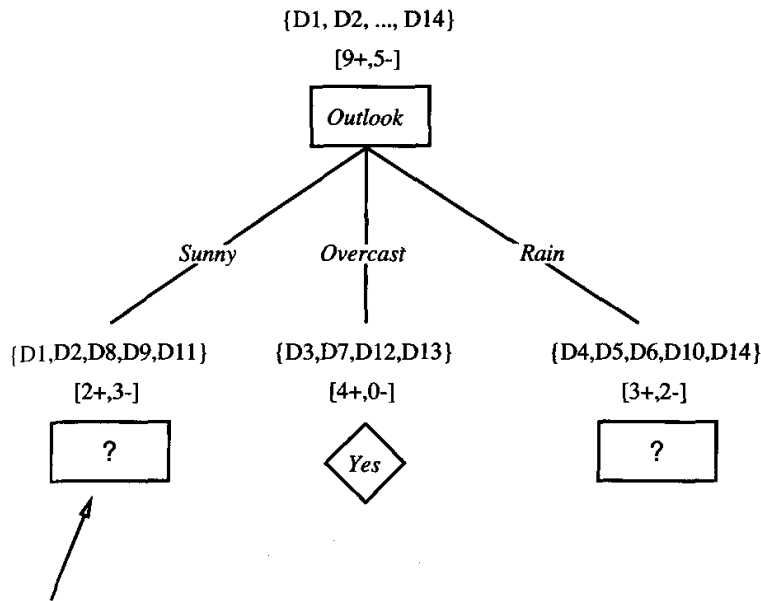
where S denotes the collection of training examples from Table 3.2.

According to the information gain measure, the *Outlook* attribute provides the best prediction of the target attribute, *PlayTennis*, over the training examples. Therefore, *Outlook* is selected as the decision attribute for the root node, and branches are created below the root for each of its possible values (i.e., *Sunny*, *Overcast*, and *Rain*). The resulting partial decision tree is shown in Figure 3.4, along with the training examples sorted to each new descendant node. Note that every example for which *Outlook* = *Overcast* is also a positive example of *PlayTennis*. Therefore, this node of the tree becomes a leaf node with the classification *PlayTennis* = *Yes*. In contrast, the descendants corresponding to *Outlook* = *Sunny* and *Outlook* = *Rain* still have nonzero entropy, and the decision tree will be further elaborated below these nodes.

The process of selecting a new attribute and partitioning the training examples is now repeated for each nonterminal descendant node, this time using only the training examples associated with that node. Attributes that have been incorporated higher in the tree are excluded, so that any given attribute can appear at most once along any path through the tree. This process continues for each new leaf node until either of two conditions is met: (1) every attribute has already been included along this path through the tree, or (2) the training examples associated with this leaf node all have the same target attribute value (i.e., their entropy is zero). Figure 3.4 illustrates the computations of information gain for the next step in growing the decision tree. The final decision tree learned by ID3 from the 14 training examples of Table 3.2 is shown in Figure 3.1.

3.5 HYPOTHESIS SPACE SEARCH IN DECISION TREE LEARNING

As with other inductive learning methods, ID3 can be characterized as searching a space of hypotheses for one that fits the training examples. The hypothesis space searched by ID3 is the set of possible decision trees. ID3 performs a simple-to-complex, hill-climbing search through this hypothesis space, beginning with the empty tree, then considering progressively more elaborate hypotheses in search of a decision tree that correctly classifies the training data. The evaluation function



Which attribute should be tested here?

$$S_{\text{sunny}} = \{D1, D2, D8, D9, D11\}$$

$$\text{Gain}(S_{\text{sunny}}, \text{Humidity}) = .970 - (3/5) 0.0 - (2/5) 0.0 = .970$$

$$\text{Gain}(S_{\text{sunny}}, \text{Temperature}) = .970 - (2/5) 0.0 - (2/5) 1.0 - (1/5) 0.0 = .570$$

$$\text{Gain}(S_{\text{sunny}}, \text{Wind}) = .970 - (2/5) 1.0 - (3/5) .918 = .019$$

FIGURE 3.4

The partially learned decision tree resulting from the first step of ID3. The training examples are sorted to the corresponding descendant nodes. The *Overcast* descendant has only positive examples and therefore becomes a leaf node with classification *Yes*. The other two nodes will be further expanded, by selecting the attribute with highest information gain relative to the new subsets of examples.

that guides this hill-climbing search is the information gain measure. This search is depicted in Figure 3.5.

By viewing ID3 in terms of its search space and search strategy, we can get some insight into its capabilities and limitations.

ID3's hypothesis space of all decision trees is a *complete* space of finite discrete-valued functions, relative to the available attributes. Because every finite discrete-valued function can be represented by some decision tree, ID3 avoids one of the major risks of methods that search incomplete hypothesis spaces (such as methods that consider only conjunctive hypotheses): that the hypothesis space might not contain the target function.

ID3 maintains only a single current hypothesis as it searches through the space of decision trees. This contrasts, for example, with the earlier version space Candidate-Elimination method, which maintains the set of *all* hypotheses consistent with the available training examples. By determining only a single hypothesis, ID3 loses the capabilities that follow from

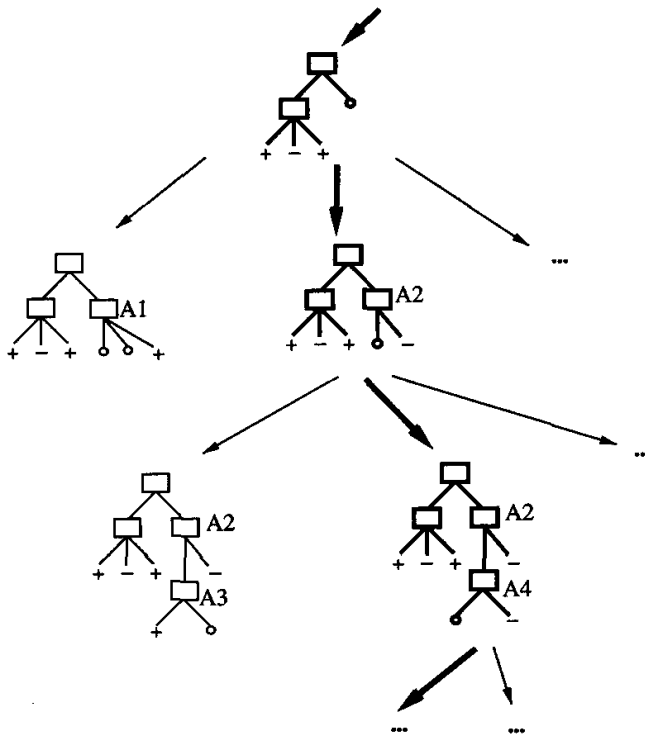


FIGURE 3.5

Hypothesis space search by ID3.
ID3 searches through the space of possible decision trees from simplest to increasingly complex, guided by the *information gain heuristic*.

explicitly representing all consistent hypotheses. For example, it does not have the ability to determine how many alternative decision trees are consistent with the available training data, or to pose new instance queries that optimally resolve among these competing hypotheses.

- ID3 in its pure form performs no backtracking in its search. Once it selects an attribute to test at a particular level in the tree, it never backtracks to reconsider this choice. Therefore, it is susceptible to the usual risks of hill-climbing search without backtracking: converging to locally optimal solutions that are not globally optimal. In the case of ID3, a locally optimal solution corresponds to the decision tree it selects along the single search path it explores. However, this locally optimal solution may be less desirable than trees that would have been encountered along a different branch of the search. Below we discuss an extension that adds a form of backtracking (post-pruning the decision tree).
- ID3 uses all training examples at each step in the search to make statistically based decisions regarding how to refine its current hypothesis. This contrasts with methods that make decisions incrementally, based on individual training examples (e.g., FIND-S or CANDIDATE-ELIMINATION). One advantage of using statistical properties of all the examples (e.g., information gain) is that the resulting search is much less sensitive to errors in individual training examples. ID3 can be easily extended to handle noisy training data by modifying its termination criterion to accept hypotheses that imperfectly fit the training data.

3.6 INDUCTIVE BIAS IN DECISION TREE LEARNING

What is the policy by which ID3 generalizes from observed training examples to classify unseen instances? In other words, what is its inductive bias? Recall from Chapter 2 that inductive bias is the set of assumptions that, together with the training data, deductively justify the classifications assigned by the learner to future instances.

Given a collection of training examples, there are typically many decision trees consistent with these examples. Describing the inductive bias of ID3 therefore consists of describing the basis by which it chooses one of these consistent hypotheses over the others. Which of these decision trees does ID3 choose? It chooses the first acceptable tree it encounters in its simple-to-complex, hill-climbing search through the space of possible trees. Roughly speaking, then, the ID3 search strategy (a) selects in favor of shorter trees over longer ones, and (b) selects trees that place the attributes with highest information gain closest to the root. Because of the subtle interaction between the attribute selection heuristic used by ID3 and the particular training examples it encounters, it is difficult to characterize precisely the inductive bias exhibited by ID3. However, we can approximately characterize its bias as a preference for short decision trees over complex trees.

Approximate inductive bias of ID3: Shorter trees are preferred over larger trees.

In fact, one could imagine an algorithm similar to ID3 that exhibits precisely this inductive bias. Consider an algorithm that begins with the empty tree and searches *breadth first* through progressively more complex trees, first considering all trees of depth 1, then all trees of depth 2, etc. Once it finds a decision tree consistent with the training data, it returns the smallest consistent tree at that search depth (e.g., the tree with the fewest nodes). Let us call this breadth-first search algorithm BFS-ID3. BFS-ID3 finds a shortest decision tree and thus exhibits precisely the bias “shorter trees are preferred over longer trees.” ID3 can be viewed as an efficient approximation to BFS-ID3, using a greedy heuristic search to attempt to find the shortest tree without conducting the entire breadth-first search through the hypothesis space.

Because ID3 uses the information gain heuristic and a hill climbing strategy, it exhibits a more complex bias than BFS-ID3. In particular, it does not always find the shortest consistent tree, and it is biased to favor trees that place attributes with high information gain closest to the root.

A closer approximation to the inductive bias of ID3: Shorter trees are preferred over longer trees. Trees that place high information gain attributes close to the root are preferred over those that do not.

3.6.1 Restriction Biases and Preference Biases

There is an interesting difference between the types of inductive bias exhibited by ID3 and by the CANDIDATE-ELIMINATION algorithm discussed in Chapter 2.

Consider the difference between the hypothesis space search in these two approaches:

- ID3 searches a *complete* hypothesis space (i.e., one capable of expressing any finite discrete-valued function). It searches *incompletely* through this space, from simple to complex hypotheses, until its termination condition is met (e.g., until it finds a hypothesis consistent with the data). Its inductive bias is solely a consequence of the ordering of hypotheses by its search strategy. Its hypothesis space introduces no additional bias.
- The version space CANDIDATE-ELIMINATION algorithm searches an *incomplete* hypothesis space (i.e., one that can express only a subset of the potentially teachable concepts). It searches this space *completely*, finding every hypothesis consistent with the training data. Its inductive bias is solely a consequence of the expressive power of its hypothesis representation. Its search strategy introduces no additional bias.

In brief, the inductive bias of ID3 follows from its *search strategy*, whereas the inductive bias of the CANDIDATE-ELIMINATION algorithm follows from the definition of its *search space*.

The inductive bias of ID3 is thus a *preference* for certain hypotheses over others (e.g., for shorter hypotheses), with no hard restriction on the hypotheses that can be eventually enumerated. This form of bias is typically called a *preference bias* (or, alternatively, a *search bias*). In contrast, the bias of the CANDIDATE-ELIMINATION algorithm is in the form of a categorical *restriction* on the set of hypotheses considered. This form of bias is typically called a *restriction bias* (or, alternatively, a *language bias*).

Given that some form of inductive bias is required in order to generalize beyond the training data (see Chapter 2), which type of inductive bias shall we prefer; a preference bias or restriction bias?

Typically, a preference bias is more desirable than a restriction bias, because it allows the learner to work within a complete hypothesis space that is assured to contain the unknown target function. In contrast, a restriction bias that strictly limits the set of potential hypotheses is generally less desirable, because it introduces the possibility of excluding the unknown target function altogether.

Whereas ID3 exhibits a purely preference bias and CANDIDATE-ELIMINATION a purely restriction bias, some learning systems combine both. Consider, for example, the program described in Chapter 1 for learning a numerical evaluation function for game playing. In this case, the learned evaluation function is represented by a linear combination of a fixed set of board features, and the learning algorithm adjusts the parameters of this linear combination to best fit the available training data. In this case, the decision to use a linear function to represent the evaluation function introduces a restriction bias (nonlinear evaluation functions cannot be represented in this form). At the same time, the choice of a particular parameter tuning method (the LMS algorithm in this case) introduces a preference bias stemming from the ordered search through the space of all possible parameter values.

3.6.2 Why Prefer Short Hypotheses?

Is ID3's inductive bias favoring shorter decision trees a sound basis for generalizing beyond the training data? Philosophers and others have debated this question for centuries, and the debate remains unresolved to this day. William of Occam was one of the first to discuss[†] the question, around the year 1320, so this bias often goes by the name of Occam's razor.

Occam's razor: Prefer the simplest hypothesis that fits the data.

Of course giving an inductive bias a name does not justify it. Why should one prefer simpler hypotheses? Notice that scientists sometimes appear to follow this inductive bias. Physicists, for example, prefer simple explanations for the motions of the planets, over more complex explanations. Why? One argument is that because there are fewer short hypotheses than long ones (based on straightforward combinatorial arguments), it is less likely that one will find a short hypothesis that coincidentally fits the training data. In contrast there are often many very complex hypotheses that fit the current training data but fail to generalize correctly to subsequent data. Consider decision tree hypotheses, for example. There are many more 500-node decision trees than 5-node decision trees. Given a small set of 20 training examples, we might expect to be able to find many 500-node decision trees consistent with these, whereas we would be more surprised if a 5-node decision tree could perfectly fit this data. We might therefore believe the 5-node tree is less likely to be a statistical coincidence and prefer this hypothesis over the 500-node hypothesis.

Upon closer examination, it turns out there is a major difficulty with the above argument. By the same reasoning we could have argued that one should prefer decision trees containing exactly 17 leaf nodes with 11 nonleaf nodes, that use the decision attribute A_1 at the root, and test attributes A_2 through A_{11} , in numerical order. There are relatively few such trees, and we might argue (by the same reasoning as above) that our a priori chance of finding one consistent with an arbitrary set of data is therefore small. The difficulty here is that there are very many small sets of hypotheses that one can define—most of them rather arcane. Why should we believe that the small set of hypotheses consisting of decision trees with *short descriptions* should be any more relevant than the multitude of other small sets of hypotheses that we might define?

A second problem with the above argument for Occam's razor is that the size of a hypothesis is determined by the particular representation used *internally* by the learner. Two learners using different internal representations could therefore arrive at different hypotheses, both justifying their contradictory conclusions by Occam's razor! For example, the function represented by the learned decision tree in Figure 3.1 could be represented as a tree with just one decision node, by a learner that uses the boolean attribute XYZ , where we define the attribute XYZ to

[†] Apparently while shaving.

be true for instances that are classified positive by the decision tree in Figure 3.1 and false otherwise. Thus, two learners, both applying Occam's razor, would generalize in different ways if one used the *XYZ* attribute to describe its examples and the other used only the attributes *Outlook*, *Temperature*, *Humidity*, and *Wind*.

This last argument shows that Occam's razor will produce two different hypotheses from the same training examples when it is applied by two learners that perceive these examples in terms of different internal representations. On this basis we might be tempted to reject Occam's razor altogether. However, consider the following scenario that examines the question of which internal representations might arise from a process of evolution and natural selection. Imagine a population of artificial learning agents created by a simulated evolutionary process involving reproduction, mutation, and natural selection of these agents. Let us assume that this evolutionary process can alter the perceptual systems of these agents from generation to generation, thereby changing the internal attributes by which they perceive their world. For the sake of argument, let us also assume that the learning agents employ a fixed learning algorithm (say ID3) that cannot be altered by evolution. It is reasonable to assume that over time evolution will produce internal representation that make these agents increasingly successful within their environment. Assuming that the success of an agent depends highly on its ability to generalize accurately, we would therefore expect evolution to develop internal representations that work well with whatever learning algorithm and inductive bias is present. If the species of agents employs a learning algorithm whose inductive bias is Occam's razor, then we expect evolution to produce internal representations for which Occam's razor is a successful strategy. The essence of the argument here is that evolution will create internal representations that make the learning algorithm's inductive bias a self-fulfilling prophecy, simply because it can alter the representation easier than it can alter the learning algorithm.

For now, we leave the debate regarding Occam's razor. We will revisit it in Chapter 6, where we discuss the Minimum Description Length principle, a version of Occam's razor that can be interpreted within a Bayesian framework.

3.7 ISSUES IN DECISION TREE LEARNING

Practical issues in learning decision trees include determining how deeply to grow the decision tree, handling continuous attributes, choosing an appropriate attribute selection measure, handling training data with missing attribute values, handling attributes with differing costs, and improving computational efficiency. Below we discuss each of these issues and extensions to the basic ID3 algorithm that address them. ID3 has itself been extended to address most of these issues, with the resulting system renamed C4.5 (Quinlan 1993).

3.7.1 Avoiding Overfitting the Data

The algorithm described in Table 3.1 grows each branch of the tree just deeply enough to perfectly classify the training examples. While this is sometimes a

reasonable strategy, in fact it can lead to difficulties when there is noise in the data, or when the number of training examples is too small to produce a representative sample of the true target function. In either of these cases, this simple algorithm can produce trees that *overfit* the training examples.

We will say that a hypothesis overfits the training examples if some other hypothesis that fits the training examples less well actually performs better over the entire distribution of instances (i.e., including instances beyond the training set).

Definition: Given a hypothesis space H , a hypothesis $h \in H$ is said to **overfit** the training data if there exists some alternative hypothesis $h' \in H$, such that h has smaller error than h' over the training examples, but h' has a smaller error than h over the entire distribution of instances.

Figure 3.6 illustrates the impact of overfitting in a typical application of decision tree learning. In this case, the ID3 algorithm is applied to the task of learning which medical patients have a form of diabetes. The horizontal axis of this plot indicates the total number of nodes in the decision tree, as the tree is being constructed. The vertical axis indicates the accuracy of predictions made by the tree. The solid line shows the accuracy of the decision tree over the training examples, whereas the broken line shows accuracy measured over an independent set of test examples (not included in the training set). Predictably, the accuracy of the tree over the training examples increases monotonically as the tree is grown. However, the accuracy measured over the independent test examples first increases, then decreases. As can be seen, once the tree size exceeds approximately 25 nodes,

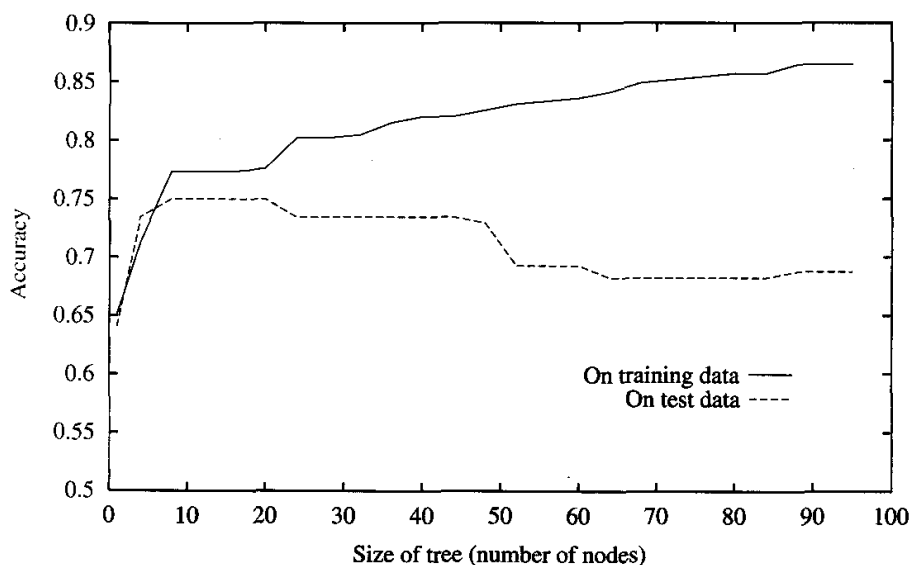


FIGURE 3.6

Overfitting in decision tree learning. As ID3 adds new nodes to grow the decision tree, the accuracy of the tree measured over the training examples increases monotonically. However, when measured over a set of test examples independent of the training examples, accuracy first increases, then decreases. Software and data for experimenting with variations on this plot are available on the World Wide Web at <http://www.cs.cmu.edu/~tom/mlbook.html>.

further elaboration of the tree decreases its accuracy over the test examples despite increasing its accuracy on the training examples.

How can it be possible for tree h to fit the training examples better than h' , but for it to perform more poorly over subsequent examples? One way this can occur is when the training examples contain random errors or noise. To illustrate, consider the effect of adding the following positive training example, incorrectly labeled as negative, to the (otherwise correct) examples in Table 3.2.

(Outlook = Sunny, Temperature = Hot, Humidity = Normal,

Wind = Strong, PlayTennis = No)

Given the original error-free data, ID3 produces the decision tree shown in Figure 3.1. However, the addition of this incorrect example will now cause ID3 to construct a more complex tree. In particular, the new example will be sorted into the second leaf node from the left in the learned tree of Figure 3.1, along with the previous positive examples D9 and D11. Because the new example is labeled as a negative example, ID3 will search for further refinements to the tree below this node. Of course as long as the new erroneous example differs in some arbitrary way from the other examples affiliated with this node, ID3 will succeed in finding a new decision attribute to separate out this new example from the two previous positive examples at this tree node. The result is that ID3 will output a decision tree (h) that is more complex than the original tree from Figure 3.1 (h'). Of course h will fit the collection of training examples perfectly, whereas the simpler h' will not. However, given that the new decision node is simply a consequence of fitting the noisy training example, we expect h to outperform h' over subsequent data drawn from the same instance distribution.

The above example illustrates how random noise in the training examples can lead to overfitting. In fact, overfitting is possible even when the training data are noise-free, especially when small numbers of examples are associated with leaf nodes. In this case, it is quite possible for coincidental regularities to occur, in which some attribute happens to partition the examples very well, despite being unrelated to the actual target function. Whenever such coincidental regularities exist, there is a risk of overfitting.

Overfitting is a significant practical difficulty for decision tree learning and many other learning methods. For example, in one experimental study of ID3 involving five different learning tasks with noisy, nondeterministic data (Mingers 1989b), overfitting was found to decrease the accuracy of learned decision trees by 10–25% on most problems.

There are several approaches to avoiding overfitting in decision tree learning. These can be grouped into two classes:

- approaches that stop growing the tree earlier, before it reaches the point where it perfectly classifies the training data,
- approaches that allow the tree to overfit the data, and then post-prune the tree.

Although the first of these approaches might seem more direct, the second approach of post-pruning overfit trees has been found to be more successful in practice. This is due to the difficulty in the first approach of estimating precisely when to stop growing the tree.

Regardless of whether the correct tree size is found by stopping early or by post-pruning, a key question is what criterion is to be used to determine the correct final tree size. Approaches include:

- Use a separate set of examples, distinct from the training examples, to evaluate the utility of post-pruning nodes from the tree.
- Use all the available data for training, but apply a statistical test to estimate whether expanding (or pruning) a particular node is likely to produce an improvement beyond the training set. For example, Quinlan (1986) uses a chi-square test to estimate whether further expanding a node is likely to improve performance over the entire instance distribution, or only on the current sample of training data.
- Use an explicit measure of the complexity for encoding the training examples and the decision tree, halting growth of the tree when this encoding size is minimized. This approach, based on a heuristic called the Minimum Description Length principle, is discussed further in Chapter 6, as well as in Quinlan and Rivest (1989) and Mehta et al. (1995).

The first of the above approaches is the most common and is often referred to as a *training and validation set* approach. We discuss the two main variants of this approach below. In this approach, the available data are separated into two sets of examples: a *training set*, which is used to form the learned hypothesis, and a separate *validation set*, which is used to evaluate the accuracy of this hypothesis over subsequent data and, in particular, to evaluate the impact of pruning this hypothesis. The motivation is this: Even though the learner may be misled by random errors and coincidental regularities within the training set, the validation set is unlikely to exhibit the same random fluctuations. Therefore, the validation set can be expected to provide a safety check against overfitting the spurious characteristics of the training set. Of course, it is important that the validation set be large enough to itself provide a statistically significant sample of the instances. One common heuristic is to withhold one-third of the available examples for the validation set, using the other two-thirds for training.

3.7.1.1 REDUCED ERROR PRUNING

How exactly might we use a validation set to prevent overfitting? One approach, called *reduced-error pruning* (Quinlan 1987), is to consider each of the decision nodes in the tree to be candidates for pruning. Pruning a decision node consists of removing the subtree rooted at that node, making it a leaf node, and assigning it the most common classification of the training examples affiliated with that node. Nodes are removed only if the resulting pruned tree performs no worse than the

original over the validation set. This has the effect that any leaf node added due to coincidental regularities in the training set is likely to be pruned because these same coincidences are unlikely to occur in the validation set. Nodes are pruned iteratively, always choosing the node whose removal most increases the decision tree accuracy over the validation set. Pruning of nodes continues until further pruning is harmful (i.e., decreases accuracy of the tree over the validation set).

The impact of reduced-error pruning on the accuracy of the decision tree is illustrated in Figure 3.7. As in Figure 3.6, the accuracy of the tree is shown measured over both training examples and test examples. The additional line in Figure 3.7 shows accuracy over the test examples as the tree is pruned. When pruning begins, the tree is at its maximum size and lowest accuracy over the test set. As pruning proceeds, the number of nodes is reduced and accuracy over the test set increases. Here, the available data has been split into three subsets: the training examples, the validation examples used for pruning the tree, and a set of test examples used to provide an unbiased estimate of accuracy over future unseen examples. The plot shows accuracy over the training and test sets. Accuracy over the validation set used for pruning is not shown.

Using a separate set of data to guide pruning is an effective approach provided a large amount of data is available. The major drawback of this approach is that when data is limited, withholding part of it for the validation set reduces even further the number of examples available for training. The following section presents an alternative approach to pruning that has been found useful in many practical situations where data is limited. Many additional techniques have been proposed as well, involving partitioning the available data several different times in

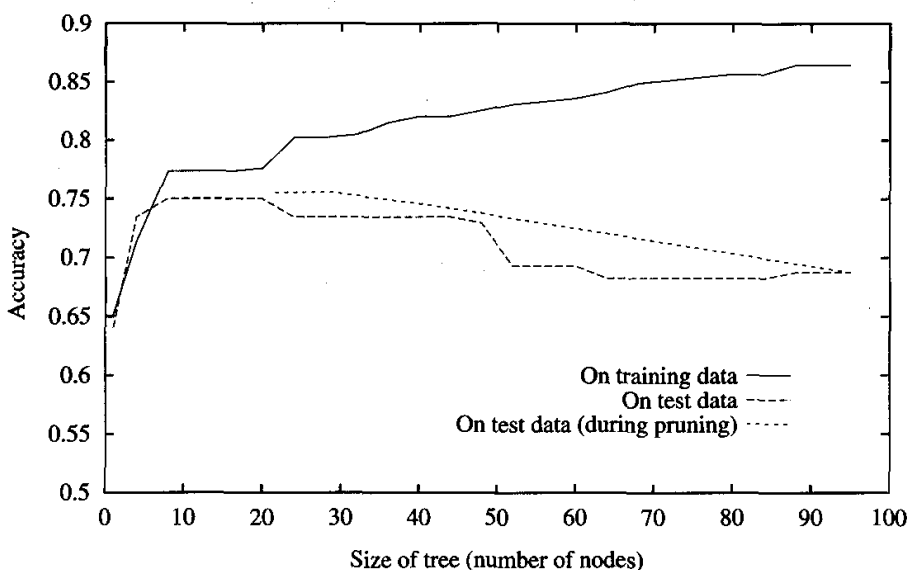


FIGURE 3.7

Effect of reduced-error pruning in decision tree learning. This plot shows the same curves of training and test set accuracy as in Figure 3.6. In addition, it shows the impact of reduced error pruning of the tree produced by ID3. Notice the increase in accuracy over the test set as nodes are pruned from the tree. Here, the validation set used for pruning is distinct from both the training and test sets.

multiple ways, then averaging the results. Empirical evaluations of alternative tree pruning methods are reported by Mingers (1989b) and by Malerba et al. (1995).

3.7.1.2 RULE POST-PRUNING

In practice, one quite successful method for finding high accuracy hypotheses is a technique we shall call *rule post-pruning*. A variant of this pruning method is used by C4.5 (Quinlan 1993), which is an outgrowth of the original ID3 algorithm. Rule post-pruning involves the following steps:

1. Infer the decision tree from the training set, growing the tree until the training data is fit as well as possible and allowing overfitting to occur.
2. Convert the learned tree into an equivalent set of rules by creating one rule for each path from the root node to a leaf node.
3. Prune (generalize) each rule by removing any preconditions that result in improving its estimated accuracy.
4. Sort the pruned rules by their estimated accuracy, and consider them in this sequence when classifying subsequent instances.

To illustrate, consider again the decision tree in Figure 3.1. In rule post-pruning, one rule is generated for each leaf node in the tree. Each attribute test along the path from the root to the leaf becomes a rule antecedent (precondition) and the classification at the leaf node becomes the rule consequent (postcondition). For example, the leftmost path of the tree in Figure 3.1 is translated into the rule

IF (*Outlook* = *Sunny*) $\wedge$ (*Humidity* = *High*)
 THEN *PlayTennis* = *No*

Next, each such rule is pruned by removing any antecedent, or precondition, whose removal does not worsen its estimated accuracy. Given the above rule, for example, rule post-pruning would consider removing the preconditions (*Outlook* = *Sunny*) and (*Humidity* = *High*). It would select whichever of these pruning steps produced the greatest improvement in estimated rule accuracy, then consider pruning the second precondition as a further pruning step. No pruning step is performed if it reduces the estimated rule accuracy.

As noted above, one method to estimate rule accuracy is to use a validation set of examples disjoint from the training set. Another method, used by C4.5, is to evaluate performance based on the training set itself, using a pessimistic estimate to make up for the fact that the training data gives an estimate biased in favor of the rules. More precisely, C4.5 calculates its pessimistic estimate by calculating the rule accuracy over the training examples to which it applies, then calculating the standard deviation in this estimated accuracy assuming a binomial distribution. For a given confidence level, the lower-bound estimate is then taken as the measure of rule performance (e.g., for a 95% confidence interval, rule accuracy is pessimistically estimated by the observed accuracy over the training

set, minus 1.96 times the estimated standard deviation). The net effect is that for large data sets, the pessimistic estimate is very close to the observed accuracy (e.g., the standard deviation is very small), whereas it grows further from the observed accuracy as the size of the data set decreases. Although this heuristic method is not statistically valid, it has nevertheless been found useful in practice. See Chapter 5 for a discussion of statistically valid approaches to estimating means and confidence intervals.

Why convert the decision tree to rules before pruning? There are three main advantages.

- Converting to rules allows distinguishing among the different contexts in which a decision node is used. Because each distinct path through the decision tree node produces a distinct rule, the pruning decision regarding that attribute test can be made differently for each path. In contrast, if the tree itself were pruned, the only two choices would be to remove the decision node completely, or to retain it in its original form.
- Converting to rules removes the distinction between attribute tests that occur near the root of the tree and those that occur near the leaves. Thus, we avoid messy bookkeeping issues such as how to reorganize the tree if the root node is pruned while retaining part of the subtree below this test.
- Converting to rules improves readability. Rules are often easier for people to understand.

3.7.2 Incorporating Continuous-Valued Attributes

Our initial definition of ID3 is restricted to attributes that take on a discrete set of values. First, the target attribute whose value is predicted by the learned tree must be discrete valued. Second, the attributes tested in the decision nodes of the tree must also be discrete valued. This second restriction can easily be removed so that continuous-valued decision attributes can be incorporated into the learned tree. This can be accomplished by dynamically defining new discrete-valued attributes that partition the continuous attribute value into a discrete set of intervals. In particular, for an attribute A that is continuous-valued, the algorithm can dynamically create a new boolean attribute A_c that is true if $A < c$ and false otherwise. The only question is how to select the best value for the threshold c .

As an example, suppose we wish to include the continuous-valued attribute *Temperature* in describing the training example days in the learning task of Table 3.2. Suppose further that the training examples associated with a particular node in the decision tree have the following values for *Temperature* and the target attribute *PlayTennis*.

<i>Temperature:</i>	40	48	60	72	80	90
<i>PlayTennis:</i>	No	No	Yes	Yes	Yes	No

What threshold-based boolean attribute should be defined based on *Temperature*? Clearly, we would like to pick a threshold, c , that produces the greatest information gain. By sorting the examples according to the continuous attribute A , then identifying adjacent examples that differ in their target classification, we can generate a set of candidate thresholds midway between the corresponding values of A . It can be shown that the value of c that maximizes information gain must always lie at such a boundary (Fayyad 1991). These candidate thresholds can then be evaluated by computing the information gain associated with each. In the current example, there are two candidate thresholds, corresponding to the values of *Temperature* at which the value of *PlayTennis* changes: $(48 + 60)/2$, and $(80 + 90)/2$. The information gain can then be computed for each of the candidate attributes, $Temperature_{>54}$ and $Temperature_{>85}$, and the best can be selected ($Temperature_{>54}$). This dynamically created boolean attribute can then compete with the other discrete-valued candidate attributes available for growing the decision tree. Fayyad and Irani (1993) discuss an extension to this approach that splits the continuous attribute into multiple intervals rather than just two intervals based on a single threshold. Utgoff and Brodley (1991) and Murthy et al. (1994) discuss approaches that define features by thresholding linear combinations of several continuous-valued attributes.

3.7.3 Alternative Measures for Selecting Attributes

There is a natural bias in the information gain measure that favors attributes with many values over those with few values. As an extreme example, consider the attribute *Date*, which has a very large number of possible values (e.g., March 4, 1979). If we were to add this attribute to the data in Table 3.2, it would have the highest information gain of any of the attributes. This is because *Date* alone perfectly predicts the target attribute over the training data. Thus, it would be selected as the decision attribute for the root node of the tree and lead to a (quite broad) tree of depth one, which perfectly classifies the training data. Of course, this decision tree would fare poorly on subsequent examples, because it is not a useful predictor despite the fact that it perfectly separates the training data.

What is wrong with the attribute *Date*? Simply put, it has so many possible values that it is bound to separate the training examples into very small subsets. Because of this, it will have a very high information gain relative to the training examples, despite being a very poor predictor of the target function over unseen instances.

One way to avoid this difficulty is to select decision attributes based on some measure other than information gain. One alternative measure that has been used successfully is the *gain ratio* (Quinlan 1986). The gain ratio measure penalizes attributes such as *Date* by incorporating a term, called *split information*, that is sensitive to how broadly and uniformly the attribute splits the data:

$$SplitInformation(S, A) \equiv - \sum_{i=1}^c \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|} \quad (3.5)$$

where S_1 through S_c are the c subsets of examples resulting from partitioning S by the c -valued attribute A . Note that *SplitInformation* is actually the entropy of S with respect to the values of attribute A . This is in contrast to our previous uses of entropy, in which we considered only the entropy of S with respect to the target attribute whose value is to be predicted by the learned tree.

The *GainRatio* measure is defined in terms of the earlier *Gain* measure, as well as this *SplitInformation*, as follows

$$\text{GainRatio}(S, A) \equiv \frac{\text{Gain}(S, A)}{\text{SplitInformation}(S, A)} \quad (3.6)$$

Notice that the *SplitInformation* term discourages the selection of attributes with many uniformly distributed values. For example, consider a collection of n examples that are completely separated by attribute A (e.g., *Date*). In this case, the *SplitInformation* value will be $\log_2 n$. In contrast, a boolean attribute B that splits the same n examples exactly in half will have *SplitInformation* of 1. If attributes A and B produce the same information gain, then clearly B will score higher according to the *GainRatio* measure.

One practical issue that arises in using *GainRatio* in place of *Gain* to select attributes is that the denominator can be zero or very small when $|S_i| \approx |S|$ for one of the S_i . This either makes the *GainRatio* undefined or very large for attributes that happen to have the same value for nearly all members of S . To avoid selecting attributes purely on this basis, we can adopt some heuristic such as first calculating the *Gain* of each attribute, then applying the *GainRatio* test only considering those attributes with above average *Gain* (Quinlan 1986).

An alternative to the *GainRatio*, designed to directly address the above difficulty, is a distance-based measure introduced by Lopez de Mantaras (1991). This measure is based on defining a distance metric between partitions of the data. Each attribute is evaluated based on the distance between the data partition it creates and the perfect partition (i.e., the partition that perfectly classifies the training data). The attribute whose partition is closest to the perfect partition is chosen. Lopez de Mantaras (1991) defines this distance measure, proves that it is not biased toward attributes with large numbers of values, and reports experimental studies indicating that the predictive accuracy of the induced trees is not significantly different from that obtained with the *Gain* and *GainRatio* measures. However, this distance measure avoids the practical difficulties associated with the *GainRatio* measure, and in his experiments it produces significantly smaller trees in the case of data sets whose attributes have very different numbers of values.

A variety of other selection measures have been proposed as well (e.g., see Breiman et al. 1984; Mingers 1989a; Kearns and Mansour 1996; Dietterich et al. 1996). Mingers (1989a) provides an experimental analysis of the relative effectiveness of several selection measures over a variety of problems. He reports significant differences in the sizes of the unpruned trees produced by the different selection measures. However, in his experimental domains the choice of attribute selection measure appears to have a smaller impact on final accuracy than does the extent and method of post-pruning.

3.7.4 Handling Training Examples with Missing Attribute Values

In certain cases, the available data may be missing values for some attributes. For example, in a medical domain in which we wish to predict patient outcome based on various laboratory tests, it may be that the lab test *Blood-Test-Result* is available only for a subset of the patients. In such cases, it is common to estimate the missing attribute value based on other examples for which this attribute has a known value.

Consider the situation in which $\text{Gain}(S, A)$ is to be calculated at node n in the decision tree to evaluate whether the attribute A is the best attribute to test at this decision node. Suppose that $\langle x, c(x) \rangle$ is one of the training examples in S and that the value $A(x)$ is unknown.

One strategy for dealing with the missing attribute value is to assign it the value that is most common among training examples at node n . Alternatively, we might assign it the most common value among examples at node n that have the classification $c(x)$. The elaborated training example using this estimated value for $A(x)$ can then be used directly by the existing decision tree learning algorithm. This strategy is examined by Mingers (1989a).

A second, more complex procedure is to assign a probability to each of the possible values of A rather than simply assigning the most common value to $A(x)$. These probabilities can be estimated again based on the observed frequencies of the various values for A among the examples at node n . For example, given a boolean attribute A , if node n contains six known examples with $A = 1$ and four with $A = 0$, then we would say the probability that $A(x) = 1$ is 0.6, and the probability that $A(x) = 0$ is 0.4. A fractional 0.6 of instance x is now distributed down the branch for $A = 1$, and a fractional 0.4 of x down the other tree branch. These fractional examples are used for the purpose of computing information Gain and can be further subdivided at subsequent branches of the tree if a second missing attribute value must be tested. This same fractioning of examples can also be applied after learning, to classify new instances whose attribute values are unknown. In this case, the classification of the new instance is simply the most probable classification, computed by summing the weights of the instance fragments classified in different ways at the leaf nodes of the tree. This method for handling missing attribute values is used in C4.5 (Quinlan 1993).

3.7.5 Handling Attributes with Differing Costs

In some learning tasks the instance attributes may have associated costs. For example, in learning to classify medical diseases we might describe patients in terms of attributes such as *Temperature*, *BiopsyResult*, *Pulse*, *BloodTestResults*, etc. These attributes vary significantly in their costs, both in terms of monetary cost and cost to patient comfort. In such tasks, we would prefer decision trees that use low-cost attributes where possible, relying on high-cost attributes only when needed to produce reliable classifications.

ID3 can be modified to take into account attribute costs by introducing a cost term into the attribute selection measure. For example, we might divide the Gain

by the cost of the attribute, so that lower-cost attributes would be preferred. While such cost-sensitive measures do not guarantee finding an optimal cost-sensitive decision tree, they do bias the search in favor of low-cost attributes.

Tan and Schlimmer (1990) and Tan (1993) describe one such approach and apply it to a robot perception task in which the robot must learn to classify different objects according to how they can be grasped by the robot's manipulator. In this case the attributes correspond to different sensor readings obtained by a movable sonar on the robot. Attribute cost is measured by the number of seconds required to obtain the attribute value by positioning and operating the sonar. They demonstrate that more efficient recognition strategies are learned, without sacrificing classification accuracy, by replacing the information gain attribute selection measure by the following measure

$$\frac{Gain^2(S, A)}{Cost(A)}$$

Nunez (1988) describes a related approach and its application to learning medical diagnosis rules. Here the attributes are different symptoms and laboratory tests with differing costs. His system uses a somewhat different attribute selection measure

$$\frac{2^{Gain(S, A)} - 1}{(Cost(A) + 1)^w}$$

where $w \in [0, 1]$ is a constant that determines the relative importance of cost versus information gain. Nunez (1991) presents an empirical comparison of these two approaches over a range of tasks.

3.8 SUMMARY AND FURTHER READING

The main points of this chapter include:

- Decision tree learning provides a practical method for concept learning and for learning other discrete-valued functions. The ID3 family of algorithms infers decision trees by growing them from the root downward, greedily selecting the next best attribute for each new decision branch added to the tree.
- ID3 searches a complete hypothesis space (i.e., the space of decision trees can represent any discrete-valued function defined over discrete-valued instances). It thereby avoids the major difficulty associated with approaches that consider only restricted sets of hypotheses: that the target function might not be present in the hypothesis space.
- The inductive bias implicit in ID3 includes a *preference* for smaller trees; that is, its search through the hypothesis space grows the tree only as large as needed in order to classify the available training examples.
- Overfitting the training data is an important issue in decision tree learning. Because the training examples are only a sample of all possible instances,

it is possible to add branches to the tree that improve performance on the training examples while decreasing performance on other instances outside this set. Methods for post-pruning the decision tree are therefore important to avoid overfitting in decision tree learning (and other inductive inference methods that employ a preference bias).

- A large variety of extensions to the basic ID3 algorithm has been developed by different researchers. These include methods for post-pruning trees, handling real-valued attributes, accommodating training examples with missing attribute values, incrementally refining decision trees as new training examples become available, using attribute selection measures other than information gain, and considering costs associated with instance attributes.

Among the earliest work on decision tree learning is Hunt's Concept Learning System (CLS) (Hunt et al. 1966) and Friedman and Breiman's work resulting in the CART system (Friedman 1977; Breiman et al. 1984). Quinlan's ID3 system (Quinlan 1979, 1983) forms the basis for the discussion in this chapter. Other early work on decision tree learning includes ASSISTANT (Kononenko et al. 1984; Cestnik et al. 1987). Implementations of decision tree induction algorithms are now commercially available on many computer platforms.

For further details on decision tree induction, an excellent book by Quinlan (1993) discusses many practical issues and provides executable code for C4.5. Mingers (1989a) and Buntine and Niblett (1992) provide two experimental studies comparing different attribute-selection measures. Mingers (1989b) and Malerba et al. (1995) provide studies of different pruning strategies. Experiments comparing decision tree learning and other learning methods can be found in numerous papers, including (Dietterich et al. 1995; Fisher and McKusick 1989; Quinlan 1988a; Shavlik et al. 1991; Thrun et al. 1991; Weiss and Kapouleas 1989).

EXERCISES

3.1. Give decision trees to represent the following boolean functions:

- (a) $A \wedge \neg B$
- (b) $A \vee [B \wedge C]$
- (c) $A \text{ XOR } B$
- (d) $[A \wedge B] \vee [C \wedge D]$

3.2. Consider the following set of training examples:

Instance	Classification	a_1	a_2
1	+	T	T
2	+	T	T
3	−	T	F
4	+	F	F
5	−	F	T
6	−	F	T

- (a) What is the entropy of this collection of training examples with respect to the target function classification?
- (b) What is the information gain of a_2 relative to these training examples?
- 3.3. True or false: If decision tree D2 is an elaboration of tree D1, then D1 is *more-general-than* D2. Assume D1 and D2 are decision trees representing arbitrary boolean functions, and that D2 is an elaboration of D1 if ID3 could extend D1 into D2. If true, give a proof; if false, a counterexample. (*More-general-than* is defined in Chapter 2.)
- 3.4. ID3 searches for just one consistent hypothesis, whereas the CANDIDATE-ELIMINATION algorithm finds all consistent hypotheses. Consider the correspondence between these two learning algorithms.
 - (a) Show the decision tree that would be learned by ID3 assuming it is given the four training examples for the *EnjoySport?* target concept shown in Table 2.1 of Chapter 2.
 - (b) What is the relationship between the learned decision tree and the version space (shown in Figure 2.3 of Chapter 2) that is learned from these same examples? Is the learned tree equivalent to one of the members of the version space?
 - (c) Add the following training example, and compute the new decision tree. This time, show the value of the information gain for each candidate attribute at each step in growing the tree.

<i>Sky</i>	<i>Air-Temp</i>	<i>Humidity</i>	<i>Wind</i>	<i>Water</i>	<i>Forecast</i>	<i>Enjoy-Sport?</i>
<i>Sunny</i>	<i>Warm</i>	<i>Normal</i>	<i>Weak</i>	<i>Warm</i>	<i>Same</i>	<i>No</i>

- (d) Suppose we wish to design a learner that (like ID3) searches a space of decision tree hypotheses and (like CANDIDATE-ELIMINATION) finds all hypotheses consistent with the data. In short, we wish to apply the CANDIDATE-ELIMINATION algorithm to searching the space of decision tree hypotheses. Show the S and G sets that result from the first training example from Table 2.1. Note S must contain the most specific decision trees consistent with the data, whereas G must contain the most general. Show how the S and G sets are refined by the second training example (you may omit syntactically distinct trees that describe the same concept). What difficulties do you foresee in applying CANDIDATE-ELIMINATION to a decision tree hypothesis space?

REFERENCES

- Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, P. J. (1984). *Classification and regression trees*. Belmont, CA: Wadsworth International Group.
- Brodley, C. E., & Utgoff, P. E. (1995). Multivariate decision trees. *Machine Learning*, 19, 45–77.
- Buntine, W., & Niblett, T. (1992). A further comparison of splitting rules for decision-tree induction. *Machine Learning*, 8, 75–86.
- Cestnik, B., Kononenko, I., & Bratko, I. (1987). ASSISTANT-86: A knowledge-elicitation tool for sophisticated users. In I. Bratko & N. Lavrač (Eds.), *Progress in machine learning*. Bled, Yugoslavia: Sigma Press.
- Dietterich, T. G., Hild, H., & Bakiri, G. (1995). A comparison of ID3 and BACKPROPAGATION for English text-to-speech mapping. *Machine Learning*, 18(1), 51–80.
- Dietterich, T. G., Kearns, M., & Mansour, Y. (1996). Applying the weak learning framework to understand and improve C4.5. *Proceedings of the 13th International Conference on Machine Learning* (pp. 96–104). San Francisco: Morgan Kaufmann.
- Fayyad, U. M. (1991). *On the induction of decision trees for multiple concept learning*, (Ph.D. dissertation). EECS Department, University of Michigan.

- Fayyad, U. M., & Irani, K. B. (1992). On the handling of continuous-valued attributes in decision tree generation. *Machine Learning*, 8, 87–102.
- Fayyad, U. M., & Irani, K. B. (1993). Multi-interval discretization of continuous-valued attributes for classification learning. In R. Bajcsy (Ed.), *Proceedings of the 13th International Joint Conference on Artificial Intelligence* (pp. 1022–1027). Morgan-Kaufmann.
- Fayyad, U. M., Weir, N., & Djorgovski, S. (1993). SKICAT: A machine learning system for automated cataloging of large scale sky surveys. *Proceedings of the Tenth International Conference on Machine Learning* (pp. 112–119). Amherst, MA: Morgan Kaufmann.
- Fisher, D. H., and McKusick, K. B. (1989). An empirical comparison of ID3 and back-propagation. *Proceedings of the Eleventh International Joint Conference on AI* (pp. 788–793). Morgan Kaufmann.
- Friedman, J. H. (1977). A recursive partitioning decision rule for non-parametric classification. *IEEE Transactions on Computers* (pp. 404–408).
- Hunt, E. B. (1975). *Artificial Intelligence*. New York: Academic Press.
- Hunt, E. B., Marin, J., & Stone, P. J. (1966). *Experiments in Induction*. New York: Academic Press.
- Kearns, M., & Mansour, Y. (1996). On the boosting ability of top-down decision tree learning algorithms. *Proceedings of the 28th ACM Symposium on the Theory of Computing*. New York: ACM Press.
- Kononenko, I., Bratko, I., & Roskar, E. (1984). *Experiments in automatic learning of medical diagnostic rules* (Technical report). Jozef Stefan Institute, Ljubljana, Yugoslavia.
- Lopez de Mantaras, R. (1991). A distance-based attribute selection measure for decision tree induction. *Machine Learning*, 6(1), 81–92.
- Malerba, D., Floriana, E., & Semeraro, G. (1995). A further comparison of simplification methods for decision tree induction. In D. Fisher & H. Lenz (Eds.), *Learning from data: AI and statistics*. Springer-Verlag.
- Mehta, M., Rissanen, J., & Agrawal, R. (1995). MDL-based decision tree pruning. *Proceedings of the First International Conference on Knowledge Discovery and Data Mining* (pp. 216–221). Menlo Park, CA: AAAI Press.
- Mingers, J. (1989a). An empirical comparison of selection measures for decision-tree induction. *Machine Learning*, 3(4), 319–342.
- Mingers, J. (1989b). An empirical comparison of pruning methods for decision-tree induction. *Machine Learning*, 4(2), 227–243.
- Murphy, P. M., & Pazzani, M. J. (1994). Exploring the decision forest: An empirical investigation of Occam's razor in decision tree induction. *Journal of Artificial Intelligence Research*, 1, 257–275.
- Murthy, S. K., Kasif, S., & Salzberg, S. (1994). A system for induction of oblique decision trees. *Journal of Artificial Intelligence Research*, 2, 1–33.
- Nunez, M. (1991). The use of background knowledge in decision tree induction. *Machine Learning*, 6(3), 231–250.
- Pagallo, G., & Haussler, D. (1990). Boolean feature discovery in empirical learning. *Machine Learning*, 5, 71–100.
- Quinlan, J. R. (1979). Discovering rules by induction from large collections of examples. In D. Michie (Ed.), *Expert systems in the micro electronic age*. Edinburgh Univ. Press.
- Quinlan, J. R. (1983). Learning efficient classification procedures and their application to chess end games. In R. S. Michalski, J. G. Carbonell, & T. M. Mitchell (Eds.), *Machine learning: An artificial intelligence approach*. San Mateo, CA: Morgan Kaufmann.
- Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.
- Quinlan, J. R. (1987). Rule induction with statistical data—a comparison with multiple regression. *Journal of the Operational Research Society*, 38, 347–352.
- Quinlan, J.R. (1988). An empirical comparison of genetic and decision-tree classifiers. *Proceedings of the Fifth International Machine Learning Conference* (135–141). San Mateo, CA: Morgan Kaufmann.
- Quinlan, J.R. (1988b). Decision trees and multi-valued attributes. In Hayes, Michie, & Richards (Eds.), *Machine Intelligence 11*, (pp. 305–318). Oxford, England: Oxford University Press.

- Quinlan, J. R., & Rivest, R. (1989). *Information and Computation*, (80), 227–248.
- Quinlan, J. R. (1993). *C4.5: Programs for Machine Learning*. San Mateo, CA: Morgan Kaufmann.
- Rissanen, J. (1983). A universal prior for integers and estimation by minimum description length. *Annals of Statistics* 11 (2), 416–431.
- Rivest, R. L. (1987). Learning decision lists. *Machine Learning*, 2(3), 229–246.
- Schaffer, C. (1993). Overfitting avoidance as bias. *Machine Learning*, 10, 113–152.
- Shavlik, J. W., Mooney, R. J., & Towell, G. G. (1991). Symbolic and neural learning algorithms: an experimental comparison. *Machine Learning*, 6(2), 111–144.
- Tan, M. (1993). Cost-sensitive learning of classification knowledge and its applications in robotics. *Machine Learning*, 13(1), 1–33.
- Tan, M., & Schlimmer, J. C. (1990). Two case studies in cost-sensitive concept acquisition. *Proceedings of the AAAI-90*.
- Thrun, S. B. et al. (1991). *The Monk's problems: A performance comparison of different learning algorithms*, (Technical report CMU-CS-91-197). Computer Science Department, Carnegie Mellon Univ., Pittsburgh, PA.
- Turney, P. D. (1995). Cost-sensitive classification: empirical evaluation of a hybrid genetic decision tree induction algorithm. *Journal of AI Research*, 2, 369–409.
- Utgoff, P. E. (1989). Incremental induction of decision trees. *Machine Learning*, 4(2), 161–186.
- Utgoff, P. E., & Brodley, C. E. (1991). *Linear machine decision trees*, (COINS Technical Report 91-10). University of Massachusetts, Amherst, MA.
- Weiss, S., & Kapouleas, I. (1989). An empirical comparison of pattern recognition, neural nets, and machine learning classification methods. *Proceedings of the Eleventh IJCAI*, (781–787), Morgan Kaufmann.

CHAPTER 4

ARTIFICIAL NEURAL NETWORKS

Artificial neural networks (ANNs) provide a general, practical method for learning real-valued, discrete-valued, and vector-valued functions from examples. Algorithms such as BACKPROPAGATION use gradient descent to tune network parameters to best fit a training set of input-output pairs. ANN learning is robust to errors in the training data and has been successfully applied to problems such as interpreting visual scenes, speech recognition, and learning robot control strategies.

4.1 INTRODUCTION

Neural network learning methods provide a robust approach to approximating real-valued, discrete-valued, and vector-valued target functions. For certain types of problems, such as learning to interpret complex real-world sensor data, artificial neural networks are among the most effective learning methods currently known. For example, the BACKPROPAGATION algorithm described in this chapter has proven surprisingly successful in many practical problems such as learning to recognize handwritten characters (LeCun et al. 1989), learning to recognize spoken words (Lang et al. 1990), and learning to recognize faces (Cottrell 1990). One survey of practical applications is provided by Rumelhart et al. (1994).

4.1.1 Biological Motivation

The study of artificial neural networks (ANNs) has been inspired in part by the observation that biological learning systems are built of very complex webs of interconnected neurons. In rough analogy, artificial neural networks are built out of a densely interconnected set of simple units, where each unit takes a number of real-valued inputs (possibly the outputs of other units) and produces a single real-valued output (which may become the input to many other units).

To develop a feel for this analogy, let us consider a few facts from neurobiology. The human brain, for example, is estimated to contain a densely interconnected network of approximately 10^{11} neurons, each connected, on average, to 10^4 others. Neuron activity is typically excited or inhibited through connections to other neurons. The fastest neuron switching times are known to be on the order of 10^{-3} seconds—quite slow compared to computer switching speeds of 10^{-10} seconds. Yet humans are able to make surprisingly complex decisions, surprisingly quickly. For example, it requires approximately 10^{-1} seconds to visually recognize your mother. Notice the sequence of neuron firings that can take place during this 10^{-1} -second interval cannot possibly be longer than a few hundred steps, given the switching speed of single neurons. This observation has led many to speculate that the information-processing abilities of biological neural systems must follow from highly parallel processes operating on representations that are distributed over many neurons. One motivation for ANN systems is to capture this kind of highly parallel computation based on distributed representations. Most ANN software runs on sequential machines emulating distributed processes, although faster versions of the algorithms have also been implemented on highly parallel machines and on specialized hardware designed specifically for ANN applications.

While ANNs are loosely motivated by biological neural systems, there are many complexities to biological neural systems that are not modeled by ANNs, and many features of the ANNs we discuss here are known to be inconsistent with biological systems. For example, we consider here ANNs whose individual units output a single constant value, whereas biological neurons output a complex time series of spikes.

Historically, two groups of researchers have worked with artificial neural networks. One group has been motivated by the goal of using ANNs to study and model biological learning processes. A second group has been motivated by the goal of obtaining highly effective machine learning algorithms, independent of whether these algorithms mirror biological processes. Within this book our interest fits the latter group, and therefore we will not dwell further on biological modeling. For more information on attempts to model biological systems using ANNs, see, for example, Churchland and Sejnowski (1992); Zornetzer et al. (1994); Gabriel and Moore (1990).

4.2 NEURAL NETWORK REPRESENTATIONS

A prototypical example of ANN learning is provided by Pomerleau's (1993) system ALVINN, which uses a learned ANN to steer an autonomous vehicle driving

at normal speeds on public highways. The input to the neural network is a 30×32 grid of pixel intensities obtained from a forward-pointed camera mounted on the vehicle. The network output is the direction in which the vehicle is steered. The ANN is trained to mimic the observed steering commands of a human driving the vehicle for approximately 5 minutes. ALVINN has used its learned networks to successfully drive at speeds up to 70 miles per hour and for distances of 90 miles on public highways (driving in the left lane of a divided public highway, with other vehicles present).

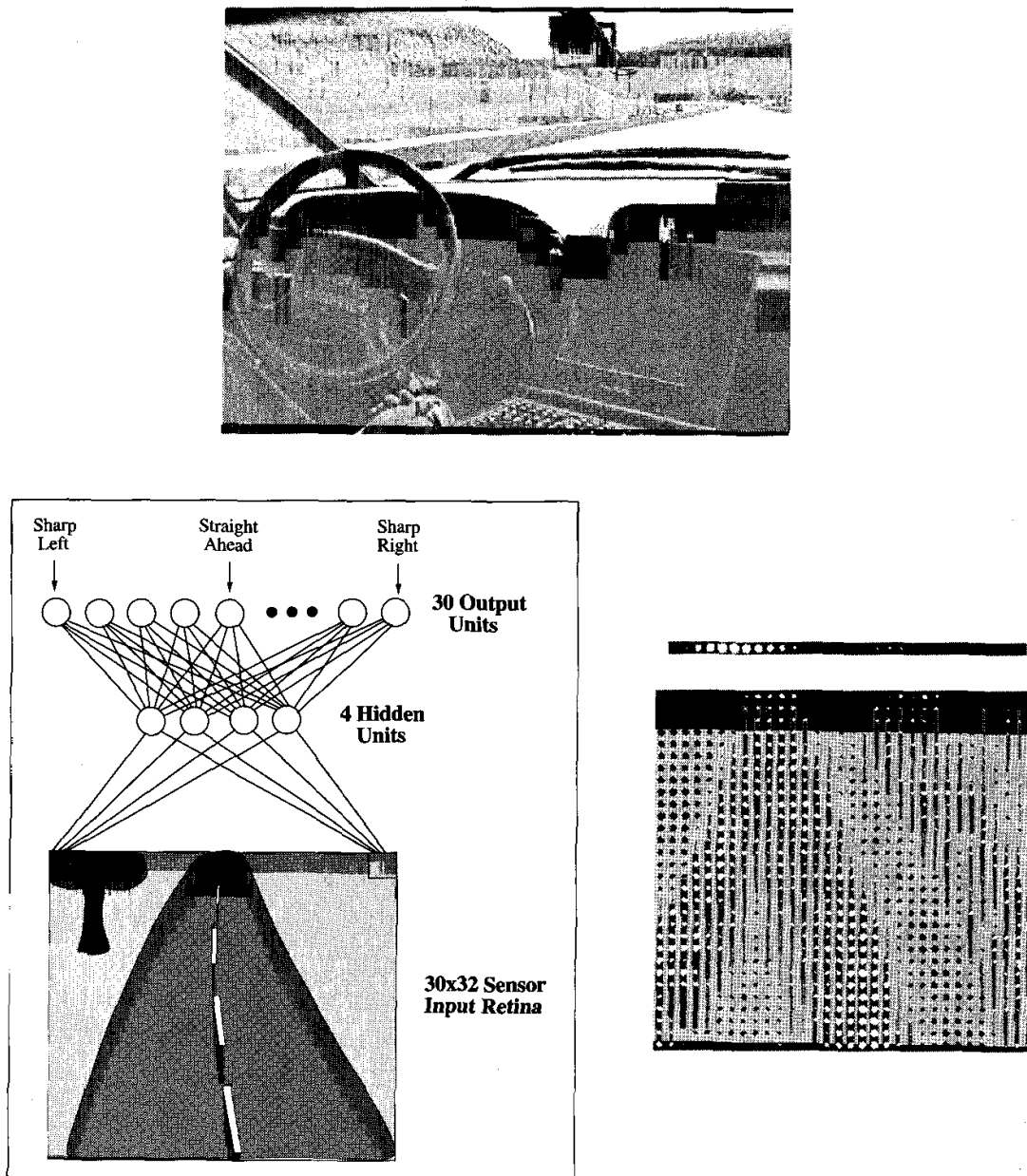
Figure 4.1 illustrates the neural network representation used in one version of the ALVINN system, and illustrates the kind of representation typical of many ANN systems. The network is shown on the left side of the figure, with the input camera image depicted below it. Each node (i.e., circle) in the network diagram corresponds to the output of a single network *unit*, and the lines entering the node from below are its inputs. As can be seen, there are four units that receive inputs directly from all of the 30×32 pixels in the image. These are called “hidden” units because their output is available only within the network and is not available as part of the global network output. Each of these four hidden units computes a single real-valued output based on a weighted combination of its 960 inputs. These hidden unit outputs are then used as inputs to a second layer of 30 “output” units. Each output unit corresponds to a particular steering direction, and the output values of these units determine which steering direction is recommended most strongly.

The diagrams on the right side of the figure depict the learned weight values associated with one of the four hidden units in this ANN. The large matrix of black and white boxes on the lower right depicts the weights from the 30×32 pixel inputs into the hidden unit. Here, a white box indicates a positive weight, a black box a negative weight, and the size of the box indicates the weight magnitude. The smaller rectangular diagram directly above the large matrix shows the weights from this hidden unit to each of the 30 output units.

The network structure of ALVINN is typical of many ANNs. Here the individual units are interconnected in layers that form a directed acyclic graph. In general, ANNs can be graphs with many types of structures—acyclic or cyclic, directed or undirected. This chapter will focus on the most common and practical ANN approaches, which are based on the BACKPROPAGATION algorithm. The BACKPROPAGATION algorithm assumes the network is a fixed structure that corresponds to a directed graph, possibly containing cycles. Learning corresponds to choosing a weight value for each edge in the graph. Although certain types of cycles are allowed, the vast majority of practical applications involve acyclic feed-forward networks, similar to the network structure used by ALVINN.

4.3 APPROPRIATE PROBLEMS FOR NEURAL NETWORK LEARNING

ANN learning is well-suited to problems in which the training data corresponds to noisy, complex sensor data, such as inputs from cameras and microphones.

**FIGURE 4.1**

Neural network learning to steer an autonomous vehicle. The ALVINN system uses BACKPROPAGATION to learn to steer an autonomous vehicle (photo at top) driving at speeds up to 70 miles per hour. The diagram on the left shows how the image of a forward-mounted camera is mapped to 960 neural network inputs, which are fed forward to 4 hidden units, connected to 30 output units. Network outputs encode the commanded steering direction. The figure on the right shows weight values for one of the hidden units in this network. The 30×32 weights into the hidden unit are displayed in the large matrix, with white blocks indicating positive and black indicating negative weights. The weights from this hidden unit to the 30 output units are depicted by the smaller rectangular block directly above the large block. As can be seen from these output weights, activation of this particular hidden unit encourages a turn toward the left.

It is also applicable to problems for which more symbolic representations are often used, such as the decision tree learning tasks discussed in Chapter 3. In these cases ANN and decision tree learning often produce results of comparable accuracy. See Shavlik et al. (1991) and Weiss and Kapouleas (1989) for experimental comparisons of decision tree and ANN learning. The BACKPROPAGATION algorithm is the most commonly used ANN learning technique. It is appropriate for problems with the following characteristics:

- *Instances are represented by many attribute-value pairs.* The target function to be learned is defined over instances that can be described by a vector of predefined features, such as the pixel values in the ALVINN example. These input attributes may be highly correlated or independent of one another. Input values can be any real values.
- *The target function output may be discrete-valued, real-valued, or a vector of several real- or discrete-valued attributes.* For example, in the ALVINN system the output is a vector of 30 attributes, each corresponding to a recommendation regarding the steering direction. The value of each output is some real number between 0 and 1, which in this case corresponds to the confidence in predicting the corresponding steering direction. We can also train a single network to output both the steering command and suggested acceleration, simply by concatenating the vectors that encode these two output predictions.
- *The training examples may contain errors.* ANN learning methods are quite robust to noise in the training data.
- *Long training times are acceptable.* Network training algorithms typically require longer training times than, say, decision tree learning algorithms. Training times can range from a few seconds to many hours, depending on factors such as the number of weights in the network, the number of training examples considered, and the settings of various learning algorithm parameters.
- *Fast evaluation of the learned target function may be required.* Although ANN learning times are relatively long, evaluating the learned network, in order to apply it to a subsequent instance, is typically very fast. For example, ALVINN applies its neural network several times per second to continually update its steering command as the vehicle drives forward.
- *The ability of humans to understand the learned target function is not important.* The weights learned by neural networks are often difficult for humans to interpret. Learned neural networks are less easily communicated to humans than learned rules.

The rest of this chapter is organized as follows: We first consider several alternative designs for the primitive units that make up artificial neural networks (perceptrons, linear units, and sigmoid units), along with learning algorithms for training single units. We then present the BACKPROPAGATION algorithm for training

multilayer networks of such units and consider several general issues such as the representational capabilities of ANNs, nature of the hypothesis space search, overfitting problems, and alternatives to the BACKPROPAGATION algorithm. A detailed example is also presented applying BACKPROPAGATION to face recognition, and directions are provided for the reader to obtain the data and code to experiment further with this application.

4.4 PERCEPTRONS

One type of ANN system is based on a unit called a *perceptron*, illustrated in Figure 4.2. A perceptron takes a vector of real-valued inputs, calculates a linear combination of these inputs, then outputs a 1 if the result is greater than some threshold and -1 otherwise. More precisely, given inputs x_1 through x_n , the output $o(x_1, \dots, x_n)$ computed by the perceptron is

$$o(x_1, \dots, x_n) = \begin{cases} 1 & \text{if } w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n > 0 \\ -1 & \text{otherwise} \end{cases}$$

where each w_i is a real-valued constant, or *weight*, that determines the contribution of input x_i to the perceptron output. Notice the quantity $(-w_0)$ is a threshold that the weighted combination of inputs $w_1x_1 + \dots + w_nx_n$ must surpass in order for the perceptron to output a 1.

To simplify notation, we imagine an additional constant input $x_0 = 1$, allowing us to write the above inequality as $\sum_{i=0}^n w_i x_i > 0$, or in vector form as $\vec{w} \cdot \vec{x} > 0$. For brevity, we will sometimes write the perceptron function as

$$o(\vec{x}) = \text{sgn}(\vec{w} \cdot \vec{x})$$

where

$$\text{sgn}(y) = \begin{cases} 1 & \text{if } y > 0 \\ -1 & \text{otherwise} \end{cases}$$

Learning a perceptron involves choosing values for the weights $w_0, \dots, w_n$. Therefore, the space H of candidate hypotheses considered in perceptron learning is the set of all possible real-valued weight vectors.

$$H = \{\vec{w} \mid \vec{w} \in \mathbb{R}^{(n+1)}\}$$

4.4.1 Representational Power of Perceptrons

We can view the perceptron as representing a hyperplane decision surface in the n -dimensional space of instances (i.e., points). The perceptron outputs a 1 for instances lying on one side of the hyperplane and outputs a -1 for instances lying on the other side, as illustrated in Figure 4.3. The equation for this decision hyperplane is $\vec{w} \cdot \vec{x} = 0$. Of course, some sets of positive and negative examples cannot be separated by any hyperplane. Those that can be separated are called *linearly separable* sets of examples.

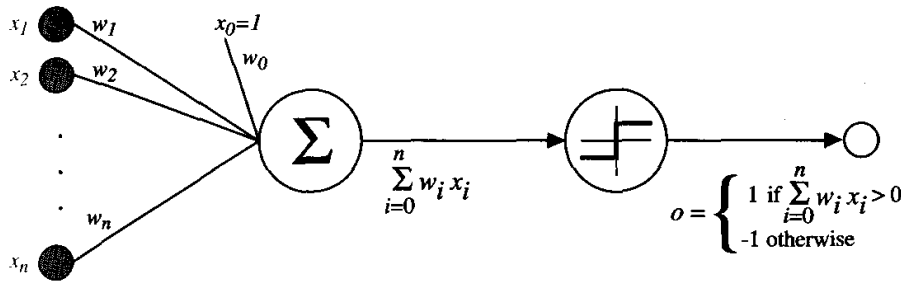


FIGURE 4.2
A perceptron.

A single perceptron can be used to represent many boolean functions. For example, if we assume boolean values of 1 (true) and -1 (false), then one way to use a two-input perceptron to implement the AND function is to set the weights $w_0 = -.8$, and $w_1 = w_2 = .5$. This perceptron can be made to represent the OR function instead by altering the threshold to $w_0 = -.3$. In fact, AND and OR can be viewed as special cases of m -of- n functions: that is, functions where at least m of the n inputs to the perceptron must be true. The OR function corresponds to $m = 1$ and the AND function to $m = n$. Any m -of- n function is easily represented using a perceptron by setting all input weights to the same value (e.g., 0.5) and then setting the threshold w_0 accordingly.

Perceptrons can represent all of the primitive boolean functions AND, OR, NAND ($\neg$ AND), and NOR ($\neg$ OR). Unfortunately, however, some boolean functions cannot be represented by a single perceptron, such as the XOR function whose value is 1 if and only if $x_1 \neq x_2$. Note the set of linearly nonseparable training examples shown in Figure 4.3(b) corresponds to this XOR function.

The ability of perceptrons to represent AND, OR, NAND, and NOR is important because *every* boolean function can be represented by some network of interconnected units based on these primitives. In fact, every boolean function can be represented by some network of perceptrons only two levels deep, in which

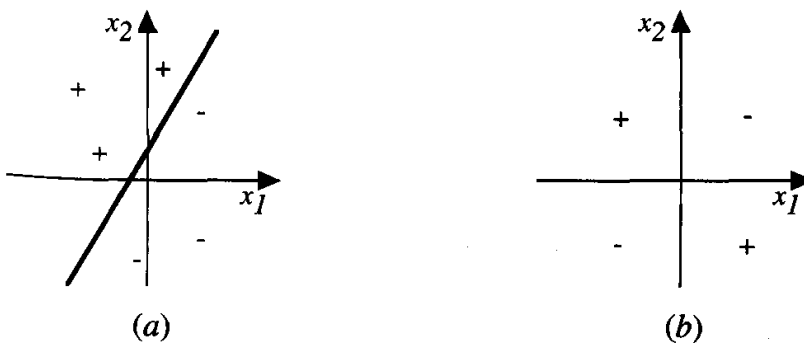


FIGURE 4.3

The decision surface represented by a two-input perceptron. (a) A set of training examples and the decision surface of a perceptron that classifies them correctly. (b) A set of training examples that is not linearly separable (i.e., that cannot be correctly classified by any straight line). x_1 and x_2 are the perceptron inputs. Positive examples are indicated by “+”, negative by “-”.

the inputs are fed to multiple units, and the outputs of these units are then input to a second, final stage. One way is to represent the boolean function in disjunctive normal form (i.e., as the disjunction (OR) of a set of conjunctions (ANDs) of the inputs and their negations). Note that the input to an AND perceptron can be negated simply by changing the sign of the corresponding input weight.

Because networks of threshold units can represent a rich variety of functions and because single units alone cannot, we will generally be interested in learning multilayer networks of threshold units.

4.4.2 The Perceptron Training Rule

Although we are interested in learning networks of many interconnected units, let us begin by understanding how to learn the weights for a single perceptron. Here the precise learning problem is to determine a weight vector that causes the perceptron to produce the correct ± 1 output for each of the given training examples.

Several algorithms are known to solve this learning problem. Here we consider two: the perceptron rule and the delta rule (a variant of the LMS rule used in Chapter 1 for learning evaluation functions). These two algorithms are guaranteed to converge to somewhat different acceptable hypotheses, under somewhat different conditions. They are important to ANNs because they provide the basis for learning networks of many units.

One way to learn an acceptable weight vector is to begin with random weights, then iteratively apply the perceptron to each training example, modifying the perceptron weights whenever it misclassifies an example. This process is repeated, iterating through the training examples as many times as needed until the perceptron classifies all training examples correctly. Weights are modified at each step according to the *perceptron training rule*, which revises the weight w_i associated with input x_i according to the rule

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = \eta(t - o)x_i$$

Here t is the target output for the current training example, o is the output generated by the perceptron, and η is a positive constant called the *learning rate*. The role of the learning rate is to moderate the degree to which weights are changed at each step. It is usually set to some small value (e.g., 0.1) and is sometimes made to decay as the number of weight-tuning iterations increases.

Why should this update rule converge toward successful weight values? To get an intuitive feel, consider some specific cases. Suppose the training example is correctly classified already by the perceptron. In this case, $(t - o)$ is zero, making Δw_i zero, so that no weights are updated. Suppose the perceptron outputs a -1 , when the target output is $+1$. To make the perceptron output a $+1$ instead of -1 in this case, the weights must be altered to increase the value of $\vec{w} \cdot \vec{x}$. For example, if $x_i > 0$, then increasing w_i will bring the perceptron closer to correctly classifying

this example. Notice the training rule will increase w_i in this case, because $(t - o)$, η , and x_i are all positive. For example, if $x_i = .8$, $\eta = 0.1$, $t = 1$, and $o = -1$, then the weight update will be $\Delta w_i = \eta(t - o)x_i = 0.1(1 - (-1))0.8 = 0.16$. On the other hand, if $t = -1$ and $o = 1$, then weights associated with positive x_i will be decreased rather than increased.

In fact, the above learning procedure can be proven to converge within a finite number of applications of the perceptron training rule to a weight vector that correctly classifies all training examples, *provided the training examples are linearly separable* and provided a sufficiently small η is used (see Minsky and Papert 1969). If the data are not linearly separable, convergence is not assured.

4.4.3 Gradient Descent and the Delta Rule

Although the perceptron rule finds a successful weight vector when the training examples are linearly separable, it can fail to converge if the examples are not linearly separable. A second training rule, called the *delta rule*, is designed to overcome this difficulty. If the training examples are not linearly separable, the delta rule converges toward a best-fit approximation to the target concept.

The key idea behind the delta rule is to use *gradient descent* to search the hypothesis space of possible weight vectors to find the weights that best fit the training examples. This rule is important because gradient descent provides the basis for the BACKPROPAGATION algorithm, which can learn networks with many interconnected units. It is also important because gradient descent can serve as the basis for learning algorithms that must search through hypothesis spaces containing many different types of continuously parameterized hypotheses.

The delta training rule is best understood by considering the task of training an *unthresholded* perceptron; that is, a *linear unit* for which the output o is given by

$$o(\vec{x}) = \vec{w} \cdot \vec{x} \quad (4.1)$$

Thus, a linear unit corresponds to the first stage of a perceptron, without the threshold.

In order to derive a weight learning rule for linear units, let us begin by specifying a measure for the *training error* of a hypothesis (weight vector), relative to the training examples. Although there are many ways to define this error, one common measure that will turn out to be especially convenient is

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \quad (4.2)$$

where D is the set of training examples, t_d is the target output for training example d , and o_d is the output of the linear unit for training example d . By this definition, $E(\vec{w})$ is simply half the squared difference between the target output t_d and the linear unit output o_d , summed over all training examples. Here we characterize E as a function of $\vec{w}$ because the linear unit output o depends on this weight vector. Of course E also depends on the particular set of training examples, but

we assume these are fixed during training, so we do not bother to write E as an explicit function of these. Chapter 6 provides a Bayesian justification for choosing this particular definition of E . In particular, there we show that under certain conditions the hypothesis that minimizes E is also the most probable hypothesis in H given the training data.

4.4.3.1 VISUALIZING THE HYPOTHESIS SPACE

To understand the gradient descent algorithm, it is helpful to visualize the entire hypothesis space of possible weight vectors and their associated E values, as illustrated in Figure 4.4. Here the axes w_0 and w_1 represent possible values for the two weights of a simple linear unit. The w_0, w_1 plane therefore represents the entire hypothesis space. The vertical axis indicates the error E relative to some fixed set of training examples. The error surface shown in the figure thus summarizes the desirability of every weight vector in the hypothesis space (we desire a hypothesis with minimum error). Given the way in which we chose to define E , for linear units this error surface must always be parabolic with a single global minimum. The specific parabola will depend, of course, on the particular set of training examples.

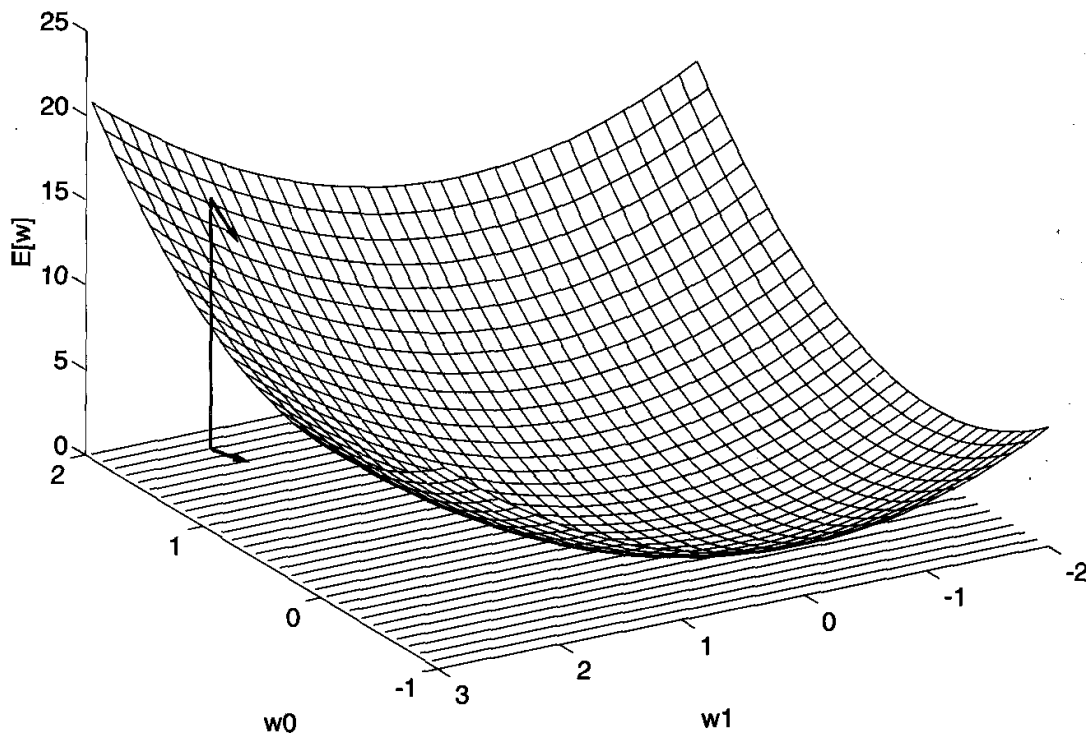


FIGURE 4.4

Error of different hypotheses. For a linear unit with two weights, the hypothesis space H is the w_0, w_1 plane. The vertical axis indicates the error of the corresponding weight vector hypothesis, relative to a fixed set of training examples. The arrow shows the negated gradient at one particular point, indicating the direction in the w_0, w_1 plane producing steepest descent along the error surface.

Gradient descent search determines a weight vector that minimizes E by starting with an arbitrary initial weight vector, then repeatedly modifying it in small steps. At each step, the weight vector is altered in the direction that produces the steepest descent along the error surface depicted in Figure 4.4. This process continues until the global minimum error is reached.

4.4.3.2 DERIVATION OF THE GRADIENT DESCENT RULE

How can we calculate the direction of steepest descent along the error surface? This direction can be found by computing the derivative of E with respect to each component of the vector $\vec{w}$. This vector derivative is called the *gradient* of E with respect to $\vec{w}$, written $\nabla E(\vec{w})$.

$$\nabla E(\vec{w}) \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n} \right] \quad (4.3)$$

Notice $\nabla E(\vec{w})$ is itself a vector, whose components are the partial derivatives of E with respect to each of the w_i . *When interpreted as a vector in weight space, the gradient specifies the direction that produces the steepest increase in E .* The negative of this vector therefore gives the direction of steepest decrease. For example, the arrow in Figure 4.4 shows the negated gradient $-\nabla E(\vec{w})$ for a particular point in the w_0, w_1 plane.

Since the gradient specifies the direction of steepest increase of E , the training rule for gradient descent is

$$\vec{w} \leftarrow \vec{w} + \Delta \vec{w}$$

where

$$\Delta \vec{w} = -\eta \nabla E(\vec{w}) \quad (4.4)$$

Here η is a positive constant called the learning rate, which determines the step size in the gradient descent search. The negative sign is present because we want to move the weight vector in the direction that *decreases* E . This training rule can also be written in its component form

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} \quad (4.5)$$

which makes it clear that steepest descent is achieved by altering each component w_i of $\vec{w}$ in proportion to $\frac{\partial E}{\partial w_i}$.

To construct a practical algorithm for iteratively updating weights according to Equation (4.5), we need an efficient way of calculating the gradient at each step. Fortunately, this is not difficult. The vector of $\frac{\partial E}{\partial w_i}$ derivatives that form the

gradient can be obtained by differentiating E from Equation (4.2), as

$$\begin{aligned}
 \frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\
 &= \frac{1}{2} \sum_{d \in D} \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\
 &= \frac{1}{2} \sum_{d \in D} 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\
 &= \sum_{d \in D} (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - \vec{w} \cdot \vec{x}_d) \\
 \frac{\partial E}{\partial w_i} &= \sum_{d \in D} (t_d - o_d)(-x_{id})
 \end{aligned} \tag{4.6}$$

where x_{id} denotes the single input component x_i for training example d . We now have an equation that gives $\frac{\partial E}{\partial w_i}$ in terms of the linear unit inputs x_{id} , outputs O_d , and target values t_d associated with the training examples. Substituting Equation (4.6) into Equation (4.5) yields the weight update rule for gradient descent

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d) x_{id} \tag{4.7}$$

To summarize, the gradient descent algorithm for training linear units is as follows: Pick an initial random weight vector. Apply the linear unit to all training examples, then compute Δw_i for each weight according to Equation (4.7). Update each weight w_i by adding Δw_i , then repeat this process. This algorithm is given in Table 4.1. Because the error surface contains only a single global minimum, this algorithm will converge to a weight vector with minimum error, regardless of whether the training examples are linearly separable, given a sufficiently small learning rate η is used. If η is too large, the gradient descent search runs the risk of overstepping the minimum in the error surface rather than settling into it. For this reason, one common modification to the algorithm is to gradually reduce the value of η as the number of gradient descent steps grows.

4.4.3.3 STOCHASTIC APPROXIMATION TO GRADIENT DESCENT

Gradient descent is an important general paradigm for learning. It is a strategy for searching through a large or infinite hypothesis space that can be applied whenever (1) the hypothesis space contains continuously parameterized hypotheses (e.g., the weights in a linear unit), and (2) the error can be differentiated with respect to these hypothesis parameters. The key practical difficulties in applying gradient descent are (1) converging to a local minimum can sometimes be quite slow (i.e., it can require many thousands of gradient descent steps), and (2) if there are multiple local minima in the error surface, then there is no guarantee that the procedure will find the global minimum.

GRADIENT-DESCENT(*training_examples*, η)

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learning rate (e.g., .05).

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
 - Initialize each Δw_i to zero.
 - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
 - Input the instance $\vec{x}$ to the unit and compute the output o
 - For each linear unit weight w_i , Do

$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i \quad (\text{T4.1})$$

- For each linear unit weight w_i , Do

$$w_i \leftarrow w_i + \Delta w_i \quad (\text{T4.2})$$

TABLE 4.1

GRADIENT DESCENT algorithm for training a linear unit. To implement the stochastic approximation to gradient descent, Equation (T4.2) is deleted, and Equation (T4.1) replaced by $w_i \leftarrow w_i + \eta(t - o)x_i$.

One common variation on gradient descent intended to alleviate these difficulties is called *incremental gradient descent*, or alternatively *stochastic gradient descent*. Whereas the gradient descent training rule presented in Equation (4.7) computes weight updates after summing over *all* the training examples in D , the idea behind stochastic gradient descent is to approximate this gradient descent search by updating weights incrementally, following the calculation of the error for *each* individual example. The modified training rule is like the training rule given by Equation (4.7) except that as we iterate through each training example we update the weight according to

$$\Delta w_i = \eta(t - o) x_i \quad (4.10)$$

where t , o , and x_i are the target value, unit output, and i th input for the training example in question. To modify the gradient descent algorithm of Table 4.1 to implement this stochastic approximation, Equation (T4.2) is simply deleted and Equation (T4.1) replaced by $w_i \leftarrow w_i + \eta(t - o) x_i$. One way to view this stochastic gradient descent is to consider a distinct error function $E_d(\vec{w})$ defined for each individual training example d as follows

$$E_d(\vec{w}) = \frac{1}{2}(t_d - o_d)^2 \quad (4.11)$$

where t_d and o_d are the target value and the unit output value for training example d . Stochastic gradient descent iterates over the training examples d in D , at each iteration altering the weights according to the gradient with respect to $E_d(\vec{w})$. The sequence of these weight updates, when iterated over all training examples, provides a reasonable approximation to descending the gradient with respect to our original error function $E(\vec{w})$. By making the value of η (the gradient

descent step size) sufficiently small, stochastic gradient descent can be made to approximate true gradient descent arbitrarily closely. The key differences between standard gradient descent and stochastic gradient descent are:

- In standard gradient descent, the error is summed over all examples before updating weights, whereas in stochastic gradient descent weights are updated upon examining each training example.
- Summing over multiple examples in standard gradient descent requires more computation per weight update step. On the other hand, because it uses the true gradient, standard gradient descent is often used with a larger step size per weight update than stochastic gradient descent.
- In cases where there are multiple local minima with respect to $E(\vec{w})$, stochastic gradient descent can sometimes avoid falling into these local minima because it uses the various $\nabla E_d(\vec{w})$ rather than $\nabla E(\vec{w})$ to guide its search.

Both stochastic and standard gradient descent methods are commonly used in practice.

The training rule in Equation (4.10) is known as the *delta rule*, or sometimes the LMS (least-mean-square) rule, Adaline rule, or Widrow-Hoff rule (after its inventors). In Chapter 1 we referred to it as the LMS weight-update rule when describing its use for learning an evaluation function for game playing. Notice the delta rule in Equation (4.10) is similar to the perceptron training rule in Equation (4.4.2). In fact, the two expressions appear to be identical. However, the rules are different because in the delta rule o refers to the linear unit output $o(\vec{x}) = \vec{w} \cdot \vec{x}$, whereas for the perceptron rule o refers to the thresholded output $o(\vec{x}) = \text{sgn}(\vec{w} \cdot \vec{x})$.

Although we have presented the delta rule as a method for learning weights for unthresholded linear units, it can easily be used to train thresholded perceptron units, as well. Suppose that $o = \vec{w} \cdot \vec{x}$ is the unthresholded linear unit output as above, and $o' = \text{sgn}(\vec{w} \cdot \vec{x})$ is the result of thresholding o as in the perceptron. Now if we wish to train a perceptron to fit training examples with target values of ± 1 for o' , we can use these same target values and examples to train o instead, using the delta rule. Clearly, if the unthresholded output o can be trained to fit these values perfectly, then the threshold output o' will fit them as well (because $\text{sgn}(1) = 1$, and $\text{sgn}(-1) = -1$). Even when the target values cannot be fit perfectly, the thresholded o' value will correctly fit the ± 1 target value whenever the linear unit output o has the correct sign. Notice, however, that while this procedure will learn weights that minimize the error in the linear unit output o , these weights will not necessarily minimize the number of training examples misclassified by the thresholded output o' .

4.4.4 Remarks

We have considered two similar algorithms for iteratively learning perceptron weights. The key difference between these algorithms is that the perceptron train-

ing rule updates weights based on the error in the *thresholded* perceptron output, whereas the delta rule updates weights based on the error in the *unthresholded* linear combination of inputs.

The difference between these two training rules is reflected in different convergence properties. The perceptron training rule converges after a finite number of iterations to a hypothesis that perfectly classifies the training data, *provided the training examples are linearly separable*. The delta rule converges only asymptotically toward the minimum error hypothesis, possibly requiring unbounded time, but converges *regardless of whether the training data are linearly separable*. A detailed presentation of the convergence proofs can be found in Hertz et al. (1991).

A third possible algorithm for learning the weight vector is linear programming. Linear programming is a general, efficient method for solving sets of linear inequalities. Notice each training example corresponds to an inequality of the form $\vec{w} \cdot \vec{x} > 0$ or $\vec{w} \cdot \vec{x} \leq 0$, and their solution is the desired weight vector. Unfortunately, this approach yields a solution only when the training examples are linearly separable; however, Duda and Hart (1973, p. 168) suggest a more subtle formulation that accommodates the nonseparable case. In any case, the approach of linear programming does not scale to training multilayer networks, which is our primary concern. In contrast, the gradient descent approach, on which the delta rule is based, can be easily extended to multilayer networks, as shown in the following section.

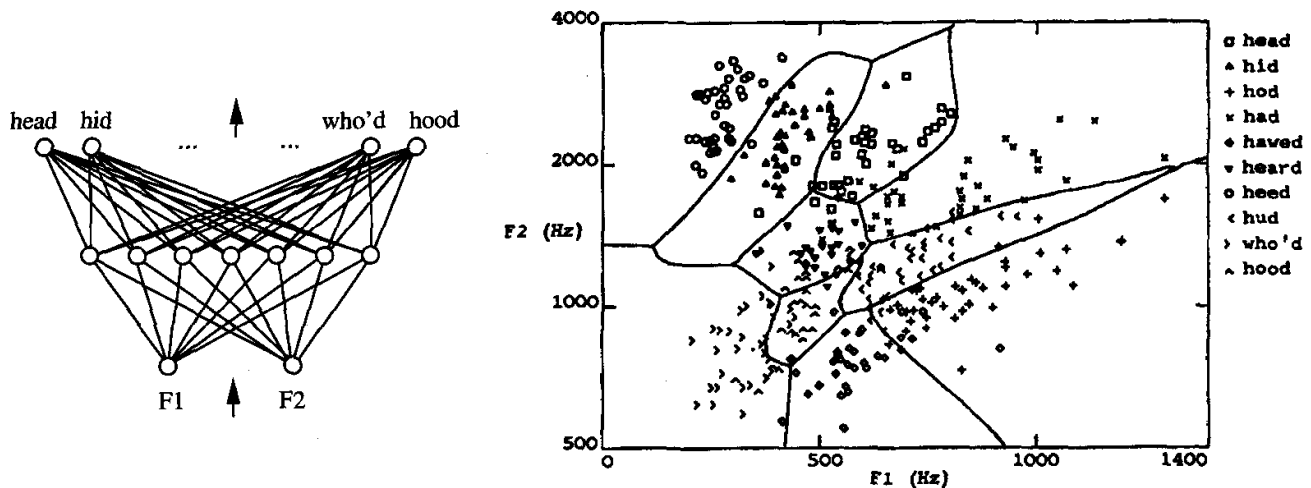
4.5 MULTILAYER NETWORKS AND THE BACKPROPAGATION ALGORITHM

As noted in Section 4.4.1, single perceptrons can only express linear decision surfaces. In contrast, the kind of multilayer networks learned by the BACKPROPAGATION algorithm are capable of expressing a rich variety of nonlinear decision surfaces. For example, a typical multilayer network and decision surface is depicted in Figure 4.5. Here the speech recognition task involves distinguishing among 10 possible vowels, all spoken in the context of “h_d” (i.e., “hid,” “had,” “head,” “hood,” etc.). The input speech signal is represented by two numerical parameters obtained from a spectral analysis of the sound, allowing us to easily visualize the decision surface over the two-dimensional instance space. As shown in the figure, it is possible for the multilayer network to represent highly nonlinear decision surfaces that are much more expressive than the linear decision surfaces of single units shown earlier in Figure 4.3.

This section discusses how to learn such multilayer networks using a gradient descent algorithm similar to that discussed in the previous section.

4.5.1 A Differentiable Threshold Unit

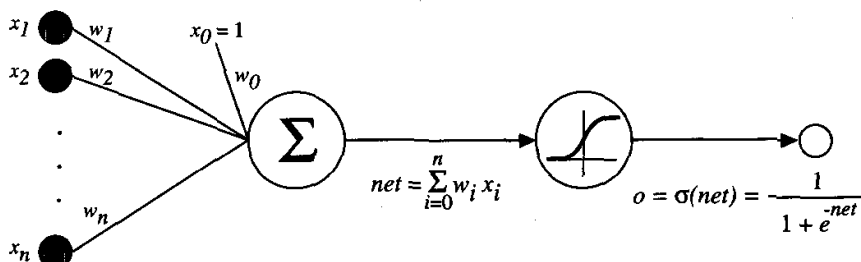
What type of unit shall we use as the basis for constructing multilayer networks? At first we might be tempted to choose the linear units discussed in the previous

**FIGURE 4.5**

Decision regions of a multilayer feedforward network. The network shown here was trained to recognize 1 of 10 vowel sounds occurring in the context “h_d” (e.g., “had,” “hid”). The network input consists of two parameters, F1 and F2, obtained from a spectral analysis of the sound. The 10 network outputs correspond to the 10 possible vowel sounds. The network prediction is the output whose value is highest. The plot on the right illustrates the highly nonlinear decision surface represented by the learned network. Points shown on the plot are test examples distinct from the examples used to train the network. (Reprinted by permission from Haung and Lippmann (1988).)

section, for which we have already derived a gradient descent learning rule. However, multiple layers of cascaded linear units still produce only linear functions, and we prefer networks capable of representing highly nonlinear functions. The perceptron unit is another possible choice, but its discontinuous threshold makes it undifferentiable and hence unsuitable for gradient descent. What we need is a unit whose output is a nonlinear function of its inputs, but whose output is also a differentiable function of its inputs. One solution is the *sigmoid unit*—a unit very much like a perceptron, but based on a smoothed, differentiable threshold function.

The sigmoid unit is illustrated in Figure 4.6. Like the perceptron, the sigmoid unit first computes a linear combination of its inputs, then applies a threshold to the result. In the case of the sigmoid unit, however, the threshold output is a

**FIGURE 4.6**

The sigmoid threshold unit.

continuous function of its input. More precisely, the sigmoid unit computes its output o as

$$o = \sigma(\vec{w} \cdot \vec{x})$$

where

$$\sigma(y) = \frac{1}{1 + e^{-y}} \quad (4.12)$$

σ is often called the sigmoid function or, alternatively, the logistic function. Note its output ranges between 0 and 1, increasing monotonically with its input (see the threshold function plot in Figure 4.6.). Because it maps a very large input domain to a small range of outputs, it is often referred to as the *squashing function* of the unit. The sigmoid function has the useful property that its derivative is easily expressed in terms of its output [in particular, $\frac{d\sigma(y)}{dy} = \sigma(y) \cdot (1 - \sigma(y))$]. As we shall see, the gradient descent learning rule makes use of this derivative. Other differentiable functions with easily calculated derivatives are sometimes used in place of σ . For example, the term e^{-y} in the sigmoid function definition is sometimes replaced by $e^{-k \cdot y}$ where k is some positive constant that determines the steepness of the threshold. The function *tanh* is also sometimes used in place of the sigmoid function (see Exercise 4.8).

4.5.2 The BACKPROPAGATION Algorithm

The BACKPROPAGATION algorithm learns the weights for a multilayer network, given a network with a fixed set of units and interconnections. It employs gradient descent to attempt to minimize the squared error between the network output values and the target values for these outputs. This section presents the BACKPROPAGATION algorithm, and the following section gives the derivation for the gradient descent weight update rule used by BACKPROPAGATION.

Because we are considering networks with multiple output units rather than single units as before, we begin by redefining E to sum the errors over all of the network output units

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2 \quad (4.13)$$

where *outputs* is the set of output units in the network, and t_{kd} and o_{kd} are the target and output values associated with the k th output unit and training example d .

The learning problem faced by BACKPROPAGATION is to search a large hypothesis space defined by all possible weight values for all the units in the network. The situation can be visualized in terms of an error surface similar to that shown for linear units in Figure 4.4. The error in that diagram is replaced by our new definition of E , and the other dimensions of the space correspond now to all of the weights associated with all of the units in the network. As in the case of training a single unit, gradient descent can be used to attempt to find a hypothesis to minimize E .

BACKPROPAGATION(*training_examples*, η , n_{in} , n_{out} , n_{hidden})

Each training example is a pair of the form $\langle \vec{x}, \vec{t} \rangle$, where $\vec{x}$ is the vector of network input values, and $\vec{t}$ is the vector of target network output values.

η is the learning rate (e.g., .05). n_{in} is the number of network inputs, n_{hidden} the number of units in the hidden layer, and n_{out} the number of output units.

The input from unit i into unit j is denoted x_{ji} , and the weight from unit i to unit j is denoted w_{ji} .

- Create a feed-forward network with n_{in} inputs, n_{hidden} hidden units, and n_{out} output units.
- Initialize all network weights to small random numbers (e.g., between $-.05$ and $.05$).
- Until the termination condition is met, Do
 - For each $\langle \vec{x}, \vec{t} \rangle$ in *training_examples*, Do

Propagate the input forward through the network:

1. Input the instance $\vec{x}$ to the network and compute the output o_u of every unit u in the network.

Propagate the errors backward through the network:

2. For each network output unit k , calculate its error term δ_k

$$\delta_k \leftarrow o_k(1 - o_k)(t_k - o_k) \quad (\text{T4.3})$$

3. For each hidden unit h , calculate its error term δ_h

$$\delta_h \leftarrow o_h(1 - o_h) \sum_{k \in \text{outputs}} w_{kh} \delta_k \quad (\text{T4.4})$$

4. Update each network weight w_{ji}

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

where

$$\Delta w_{ji} = \eta \delta_j x_{ji} \quad (\text{T4.5})$$

TABLE 4.2

The stochastic gradient descent version of the BACKPROPAGATION algorithm for feedforward networks containing two layers of sigmoid units.

One major difference in the case of multilayer networks is that the error surface can have multiple local minima, in contrast to the single-minimum parabolic error surface shown in Figure 4.4. Unfortunately, this means that gradient descent is guaranteed only to converge toward some local minimum, and not necessarily the global minimum error. Despite this obstacle, in practice BACKPROPAGATION has been found to produce excellent results in many real-world applications.

The BACKPROPAGATION algorithm is presented in Table 4.2. The algorithm as described here applies to layered feedforward networks containing two layers of sigmoid units, with units at each layer connected to all units from the preceding layer. This is the incremental, or stochastic, gradient descent version of BACKPROPAGATION. The notation used here is the same as that used in earlier sections, with the following extensions:

- An index (e.g., an integer) is assigned to each node in the network, where a “node” is either an input to the network or the output of some unit in the network.
- x_{ji} denotes the input from node i to unit j , and w_{ji} denotes the corresponding weight.
- δ_n denotes the error term associated with unit n . It plays a role analogous to the quantity $(t - o)$ in our earlier discussion of the delta training rule. As we shall see later, $\delta_n = -\frac{\partial E}{\partial net_n}$.

Notice the algorithm in Table 4.2 begins by constructing a network with the desired number of hidden and output units and initializing all network weights to small random values. Given this fixed network structure, the main loop of the algorithm then repeatedly iterates over the training examples. For each training example, it applies the network to the example, calculates the error of the network output for this example, computes the gradient with respect to the error on this example, then updates all weights in the network. This gradient descent step is iterated (often thousands of times, using the same training examples multiple times) until the network performs acceptably well.

The gradient descent weight-update rule (Equation [T4.5] in Table 4.2) is similar to the delta training rule (Equation [4.10]). Like the delta rule, it updates each weight in proportion to the learning rate η , the input value x_{ji} to which the weight is applied, and the error in the output of the unit. The only difference is that the error $(t - o)$ in the delta rule is replaced by a more complex error term, δ_j . The exact form of δ_j follows from the derivation of the weight-tuning rule given in Section 4.5.3. To understand it intuitively, first consider how δ_k is computed for each network *output* unit k (Equation [T4.3] in the algorithm). δ_k is simply the familiar $(t_k - o_k)$ from the delta rule, multiplied by the factor $o_k(1 - o_k)$, which is the derivative of the sigmoid squashing function. The δ_h value for each *hidden* unit h has a similar form (Equation [T4.4] in the algorithm). However, since training examples provide target values t_k *only* for network outputs, no target values are directly available to indicate the error of hidden units' values. Instead, the error term for hidden unit h is calculated by summing the error terms δ_k for each output unit influenced by h , weighting each of the δ_k 's by w_{kh} , the weight from hidden unit h to output unit k . This weight characterizes the degree to which hidden unit h is “responsible for” the error in output unit k .

The algorithm in Table 4.2 updates weights incrementally, following the presentation of each training example. This corresponds to a stochastic approximation to gradient descent. To obtain the true gradient of E one would sum the $\delta_j x_{ji}$ values over all training examples before altering weight values.

The weight-update loop in BACKPROPAGATION may be iterated thousands of times in a typical application. A variety of termination conditions can be used to halt the procedure. One may choose to halt after a fixed number of iterations through the loop, or once the error on the training examples falls below some threshold, or once the error on a separate validation set of examples meets some

criterion. The choice of termination criterion is an important one, because too few iterations can fail to reduce error sufficiently, and too many can lead to overfitting the training data. This issue is discussed in greater detail in Section 4.6.5.

4.5.2.1 ADDING MOMENTUM

Because BACKPROPAGATION is such a widely used algorithm, many variations have been developed. Perhaps the most common is to alter the weight-update rule in Equation (T4.5) in the algorithm by making the weight update on the n th iteration depend partially on the update that occurred during the $(n - 1)$ th iteration, as follows:

$$\Delta w_{ji}(n) = \eta \delta_j x_{ji} + \alpha \Delta w_{ji}(n-1) \quad (4.18)$$

Here $\Delta w_{ji}(n)$ is the weight update performed during the n th iteration through the main loop of the algorithm, and $0 \leq \alpha < 1$ is a constant called the *momentum*. Notice the first term on the right of this equation is just the weight-update rule of Equation (T4.5) in the BACKPROPAGATION algorithm. The second term on the right is new and is called the momentum term. To see the effect of this momentum term, consider that the gradient descent search trajectory is analogous to that of a (momentumless) ball rolling down the error surface. The effect of α is to add momentum that tends to keep the ball rolling in the same direction from one iteration to the next. This can sometimes have the effect of keeping the ball rolling through small local minima in the error surface, or along flat regions in the surface where the ball would stop if there were no momentum. It also has the effect of gradually increasing the step size of the search in regions where the gradient is unchanging, thereby speeding convergence.

4.5.2.2 LEARNING IN ARBITRARY ACYCLIC NETWORKS

The definition of BACKPROPAGATION presented in Table 4.2 applies only to two-layer networks. However, the algorithm given there easily generalizes to feedforward networks of arbitrary depth. The weight update rule seen in Equation (T4.5) is retained, and the only change is to the procedure for computing δ values. In general, the δ_r value for a unit r in layer m is computed from the δ values at the next deeper layer $m + 1$ according to

$$\delta_r = o_r (1 - o_r) \sum_{s \in \text{layer } m+1} w_{sr} \delta_s \quad (4.19)$$

Notice this is identical to Step 3 in the algorithm of Table 4.2, so all we are really saying here is that this step may be repeated for any number of hidden layers in the network.

It is equally straightforward to generalize the algorithm to any directed acyclic graph, regardless of whether the network units are arranged in uniform layers as we have assumed up to now. In the case that they are not, the rule for calculating δ for any internal unit (i.e., any unit that is not an output) is

$$\delta_r = o_r (1 - o_r) \sum_{s \in \text{Downstream}(r)} w_{sr} \delta_s \quad (4.20)$$

where $Downstream(r)$ is the set of units immediately downstream from unit r in the network: that is, all units whose inputs include the output of unit r . It is this general form of the weight-update rule that we derive in Section 4.5.3.

4.5.3 Derivation of the BACKPROPAGATION Rule

This section presents the derivation of the BACKPROPAGATION weight-tuning rule. It may be skipped on a first reading, without loss of continuity.

The specific problem we address here is deriving the stochastic gradient descent rule implemented by the algorithm in Table 4.2. Recall from Equation (4.11) that stochastic gradient descent involves iterating through the training examples one at a time, for each training example d descending the gradient of the error E_d with respect to this single example. In other words, for each training example d every weight w_{ji} is updated by adding to it Δw_{ji}

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} \quad (4.21)$$

where E_d is the error on training example d , summed over all output units in the network

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$

Here outputs is the set of output units in the network, t_k is the target value of unit k for training example d , and o_k is the output of unit k given training example d .

The derivation of the stochastic gradient descent rule is conceptually straightforward, but requires keeping track of a number of subscripts and variables. We will follow the notation shown in Figure 4.6, adding a subscript j to denote to the j th unit of the network as follows:

- x_{ji} = the i th input to unit j
- w_{ji} = the weight associated with the i th input to unit j
- $net_j = \sum_i w_{ji}x_{ji}$ (the weighted sum of inputs for unit j)
- o_j = the output computed by unit j
- t_j = the target output for unit j
- σ = the sigmoid function
- outputs = the set of units in the final layer of the network
- $Downstream(j)$ = the set of units whose immediate inputs include the output of unit j

We now derive an expression for $\frac{\partial E_d}{\partial w_{ji}}$ in order to implement the stochastic gradient descent rule seen in Equation (4.21). To begin, notice that weight w_{ji} can influence the rest of the network only through net_j . Therefore, we can use the

chain rule to write

$$\begin{aligned}\frac{\partial E_d}{\partial w_{ji}} &= \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}} \\ &= \frac{\partial E_d}{\partial net_j} x_{ji}\end{aligned}\quad (4.22)$$

Given Equation (4.22), our remaining task is to derive a convenient expression for $\frac{\partial E_d}{\partial net_j}$. We consider two cases in turn: the case where unit j is an output unit for the network, and the case where j is an internal unit.

Case 1: Training Rule for Output Unit Weights. Just as w_{ji} can influence the rest of the network only through net_j , net_j can influence the network only through o_j . Therefore, we can invoke the chain rule again to write

$$\frac{\partial E_d}{\partial net_j} = \frac{\partial E_d}{\partial o_j} \frac{\partial o_j}{\partial net_j} \quad (4.23)$$

To begin, consider just the first term in Equation (4.23)

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$

The derivatives $\frac{\partial}{\partial o_j} (t_k - o_k)^2$ will be zero for all output units k except when $k = j$. We therefore drop the summation over output units and simply set $k = j$.

$$\begin{aligned}\frac{\partial E_d}{\partial o_j} &= \frac{\partial}{\partial o_j} \frac{1}{2} (t_j - o_j)^2 \\ &= \frac{1}{2} 2(t_j - o_j) \frac{\partial (t_j - o_j)}{\partial o_j} \\ &= -(t_j - o_j)\end{aligned}\quad (4.24)$$

Next consider the second term in Equation (4.23). Since $o_j = \sigma(net_j)$, the derivative $\frac{\partial o_j}{\partial net_j}$ is just the derivative of the sigmoid function, which we have already noted is equal to $\sigma(net_j)(1 - \sigma(net_j))$. Therefore,

$$\begin{aligned}\frac{\partial o_j}{\partial net_j} &= \frac{\partial \sigma(net_j)}{\partial net_j} \\ &= o_j(1 - o_j)\end{aligned}\quad (4.25)$$

Substituting expressions (4.24) and (4.25) into (4.23), we obtain

$$\frac{\partial E_d}{\partial net_j} = -(t_j - o_j) o_j(1 - o_j) \quad (4.26)$$

and combining this with Equations (4.21) and (4.22), we have the stochastic gradient descent rule for output units

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta (t_j - o_j) o_j (1 - o_j) x_{ji} \quad (4.27)$$

Note this training rule is exactly the weight update rule implemented by Equations (T4.3) and (T4.5) in the algorithm of Table 4.2. Furthermore, we can see now that δ_k in Equation (T4.3) is equal to the quantity $-\frac{\partial E_d}{\partial net_k}$. In the remainder of this section we will use δ_i to denote the quantity $-\frac{\partial E_d}{\partial net_i}$ for an arbitrary unit i .

Case 2: Training Rule for Hidden Unit Weights. In the case where j is an internal, or hidden unit in the network, the derivation of the training rule for w_{ji} must take into account the indirect ways in which w_{ji} can influence the network outputs and hence E_d . For this reason, we will find it useful to refer to the set of all units immediately downstream of unit j in the network (i.e., all units whose direct inputs include the output of unit j). We denote this set of units by $Downstream(j)$. Notice that net_j can influence the network outputs (and therefore E_d) only through the units in $Downstream(j)$. Therefore, we can write

$$\begin{aligned} \frac{\partial E_d}{\partial net_j} &= \sum_{k \in Downstream(j)} \frac{\partial E_d}{\partial net_k} \frac{\partial net_k}{\partial net_j} \\ &= \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial net_j} \\ &= \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial o_j} \frac{\partial o_j}{\partial net_j} \\ &= \sum_{k \in Downstream(j)} -\delta_k w_{kj} \frac{\partial o_j}{\partial net_j} \\ &= \sum_{k \in Downstream(j)} -\delta_k w_{kj} o_j (1 - o_j) \end{aligned} \quad (4.28)$$

Rearranging terms and using δ_j to denote $-\frac{\partial E_d}{\partial net_j}$, we have

$$\delta_j = o_j (1 - o_j) \sum_{k \in Downstream(j)} \delta_k w_{kj}$$

and

$$\Delta w_{ji} = \eta \delta_j x_{ji}$$

which is precisely the general rule from Equation (4.20) for updating internal unit weights in arbitrary acyclic directed graphs. Notice Equation (T4.4) from Table 4.2 is just a special case of this rule, in which $Downstream(j) = outputs$.

4.6 REMARKS ON THE BACKPROPAGATION ALGORITHM

4.6.1 Convergence and Local Minima

As shown above, the BACKPROPAGATION algorithm implements a gradient descent search through the space of possible network weights, iteratively reducing the error E between the training example target values and the network outputs. Because the error surface for multilayer networks may contain many different local minima, gradient descent can become trapped in any of these. As a result, BACKPROPAGATION over multilayer networks is only guaranteed to converge toward some local minimum in E and not necessarily to the global minimum error.

Despite the lack of assured convergence to the global minimum error, BACKPROPAGATION is a highly effective function approximation method in practice. In many practical applications the problem of local minima has not been found to be as severe as one might fear. To develop some intuition here, consider that networks with large numbers of weights correspond to error surfaces in very high dimensional spaces (one dimension per weight). When gradient descent falls into a local minimum with respect to one of these weights, it will not necessarily be in a local minimum with respect to the other weights. In fact, the more weights in the network, the more dimensions that might provide “escape routes” for gradient descent to fall away from the local minimum with respect to this single weight.

A second perspective on local minima can be gained by considering the manner in which network weights evolve as the number of training iterations increases. Notice that if network weights are initialized to values near zero, then during early gradient descent steps the network will represent a very smooth function that is approximately linear in its inputs. This is because the sigmoid threshold function itself is approximately linear when the weights are close to zero (see the plot of the sigmoid function in Figure 4.6). Only after the weights have had time to grow will they reach a point where they can represent highly nonlinear network functions. One might expect more local minima to exist in the region of the weight space that represents these more complex functions. One hopes that by the time the weights reach this point they have already moved close enough to the global minimum that even local minima in this region are acceptable.

Despite the above comments, gradient descent over the complex error surfaces represented by ANNs is still poorly understood, and no methods are known to predict with certainty when local minima will cause difficulties. Common heuristics to attempt to alleviate the problem of local minima include:

- Add a momentum term to the weight-update rule as described in Equation (4.18). Momentum can sometimes carry the gradient descent procedure through narrow local minima (though in principle it can also carry it through narrow global minima into other local minima!).
- Use stochastic gradient descent rather than true gradient descent. As discussed in Section 4.4.3.3, the stochastic approximation to gradient descent effectively descends a different error surface for each training example, re-

lying on the average of these to approximate the gradient with respect to the full training set. These different error surfaces typically will have different local minima, making it less likely that the process will get stuck in any one of them.

- Train multiple networks using the same data, but initializing each network with different random weights. If the different training efforts lead to different local minima, then the network with the best performance over a separate validation data set can be selected. Alternatively, all networks can be retained and treated as a “committee” of networks whose output is the (possibly weighted) average of the individual network outputs.

4.6.2 Representational Power of Feedforward Networks

What set of functions can be represented by feedforward networks? Of course the answer depends on the width and depth of the networks. Although much is still unknown about which function classes can be described by which types of networks, three quite general results are known:

- *Boolean functions.* Every boolean function can be represented exactly by some network with two layers of units, although the number of hidden units required grows exponentially in the worst case with the number of network inputs. To see how this can be done, consider the following general scheme for representing an arbitrary boolean function: For each possible input vector, create a distinct hidden unit and set its weights so that it activates if and only if this specific vector is input to the network. This produces a hidden layer that will always have exactly one unit active. Now implement the output unit as an OR gate that activates just for the desired input patterns.
- *Continuous functions.* Every bounded continuous function can be approximated with arbitrarily small error (under a finite norm) by a network with two layers of units (Cybenko 1989; Hornik et al. 1989). The theorem in this case applies to networks that use sigmoid units at the hidden layer and (unthresholded) linear units at the output layer. The number of hidden units required depends on the function to be approximated.
- *Arbitrary functions.* Any function can be approximated to arbitrary accuracy by a network with three layers of units (Cybenko 1988). Again, the output layer uses linear units, the two hidden layers use sigmoid units, and the number of units required at each layer is not known in general. The proof of this involves showing that any function can be approximated by a linear combination of many localized functions that have value 0 everywhere except for some small region, and then showing that two layers of sigmoid units are sufficient to produce good local approximations.

These results show that limited depth feedforward networks provide a very expressive hypothesis space for BACKPROPAGATION. However, it is important to

keep in mind that the network weight vectors reachable by gradient descent from the initial weight values may not include all possible weight vectors. Hertz et al. (1991) provide a more detailed discussion of the above results.

4.6.3 Hypothesis Space Search and Inductive Bias

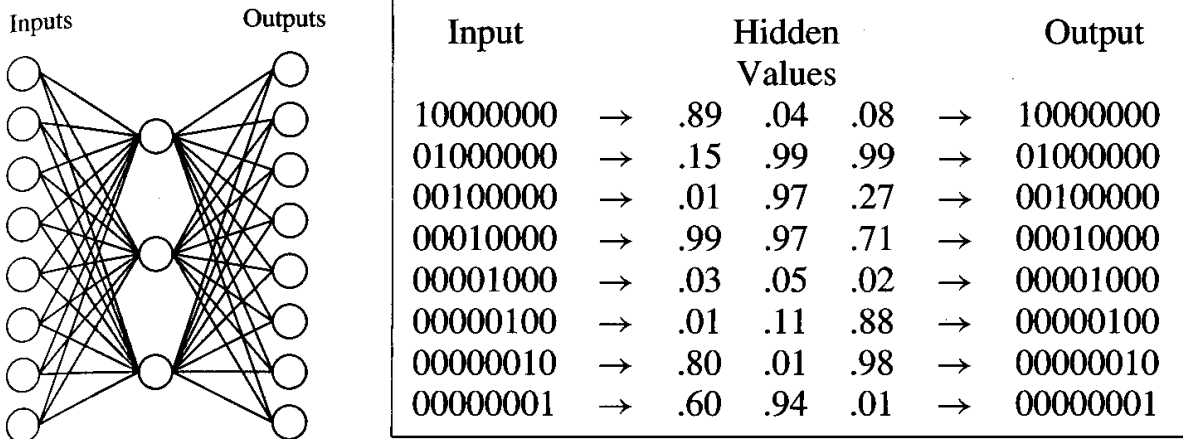
It is interesting to compare the hypothesis space search of BACKPROPAGATION to the search performed by other learning algorithms. For BACKPROPAGATION, every possible assignment of network weights represents a syntactically distinct hypothesis that in principle can be considered by the learner. In other words, the hypothesis space is the n -dimensional Euclidean space of the n network weights. Notice this hypothesis space is *continuous*, in contrast to the hypothesis spaces of decision tree learning and other methods based on discrete representations. The fact that it is continuous, together with the fact that E is differentiable with respect to the continuous parameters of the hypothesis, results in a well-defined error gradient that provides a very useful structure for organizing the search for the best hypothesis. This structure is quite different from the general-to-specific ordering used to organize the search for symbolic concept learning algorithms, or the simple-to-complex ordering over decision trees used by the ID3 and C4.5 algorithms.

What is the inductive bias by which BACKPROPAGATION generalizes beyond the observed data? It is difficult to characterize precisely the inductive bias of BACKPROPAGATION learning, because it depends on the interplay between the gradient descent search and the way in which the weight space spans the space of representable functions. However, one can roughly characterize it as *smooth interpolation between data points*. Given two positive training examples with no negative examples between them, BACKPROPAGATION will tend to label points in between as positive examples as well. This can be seen, for example, in the decision surface illustrated in Figure 4.5, in which the specific sample of training examples gives rise to smoothly varying decision regions.

4.6.4 Hidden Layer Representations

One intriguing property of BACKPROPAGATION is its ability to discover useful intermediate representations at the hidden unit layers inside the network. Because training examples constrain only the network inputs and outputs, the weight-tuning procedure is free to set weights that define whatever hidden unit representation is most effective at minimizing the squared error E . This can lead BACKPROPAGATION to define new hidden layer features that are not explicit in the input representation, but which capture properties of the input instances that are most relevant to learning the target function.

Consider, for example, the network shown in Figure 4.7. Here, the eight network inputs are connected to three hidden units, which are in turn connected to the eight output units. Because of this structure, the three hidden units will be forced to re-represent the eight input values in some way that captures their

**FIGURE 4.7**

Learned Hidden Layer Representation. This $8 \times 3 \times 8$ network was trained to learn the identity function, using the eight training examples shown. After 5000 training epochs, the three hidden unit values encode the eight distinct inputs using the encoding shown on the right. Notice if the encoded values are rounded to zero or one, the result is the standard binary encoding for eight distinct values.

relevant features, so that this hidden layer representation can be used by the output units to compute the correct target values.

Consider training the network shown in Figure 4.7 to learn the simple target function $f(\vec{x}) = \vec{x}$, where $\vec{x}$ is a vector containing seven 0's and a single 1. The network must learn to reproduce the eight inputs at the corresponding eight output units. Although this is a simple function, the network in this case is constrained to use only three hidden units. Therefore, the essential information from all eight input units must be captured by the three learned hidden units.

When BACKPROPAGATION is applied to this task, using each of the eight possible vectors as training examples, it successfully learns the target function. What hidden layer representation is created by the gradient descent BACKPROPAGATION algorithm? By examining the hidden unit values generated by the learned network for each of the eight possible input vectors, it is easy to see that the learned encoding is similar to the familiar standard binary encoding of eight values using three bits (e.g., 000, 001, 010, ..., 111). The exact values of the hidden units for one typical run of BACKPROPAGATION are shown in Figure 4.7.

This ability of multilayer networks to automatically discover useful representations at the hidden layers is a key feature of ANN learning. In contrast to learning methods that are constrained to use only predefined features provided by the human designer, this provides an important degree of flexibility that allows the learner to invent features not explicitly introduced by the human designer. Of course these invented features must still be computable as sigmoid unit functions of the provided network inputs. Note when more layers of units are used in the network, more complex features can be invented. Another example of hidden layer features is provided in the face recognition application discussed in Section 4.7.

In order to develop a better intuition for the operation of BACKPROPAGATION in this example, let us examine the operation of the gradient descent procedure in

greater detail[†]. The network in Figure 4.7 was trained using the algorithm shown in Table 4.2, with initial weights set to random values in the interval $(-0.1, 0.1)$, learning rate $\eta = 0.3$, and no weight momentum (i.e., $\alpha = 0$). Similar results were obtained by using other learning rates and by including nonzero momentum. The hidden unit encoding shown in Figure 4.7 was obtained after 5000 training iterations through the outer loop of the algorithm (i.e., 5000 iterations through each of the eight training examples). Most of the interesting weight changes occurred, however, during the first 2500 iterations.

We can directly observe the effect of BACKPROPAGATION's gradient descent search by plotting the squared output error as a function of the number of gradient descent search steps. This is shown in the top plot of Figure 4.8. Each line in this plot shows the squared output error summed over all training examples, for one of the eight network outputs. The horizontal axis indicates the number of iterations through the outermost loop of the BACKPROPAGATION algorithm. As this plot indicates, the sum of squared errors for each output decreases as the gradient descent procedure proceeds, more quickly for some output units and less quickly for others.

The evolution of the hidden layer representation can be seen in the second plot of Figure 4.8. This plot shows the three hidden unit values computed by the learned network for one of the possible inputs (in particular, 01000000). Again, the horizontal axis indicates the number of training iterations. As this plot indicates, the network passes through a number of different encodings before converging to the final encoding given in Figure 4.7.

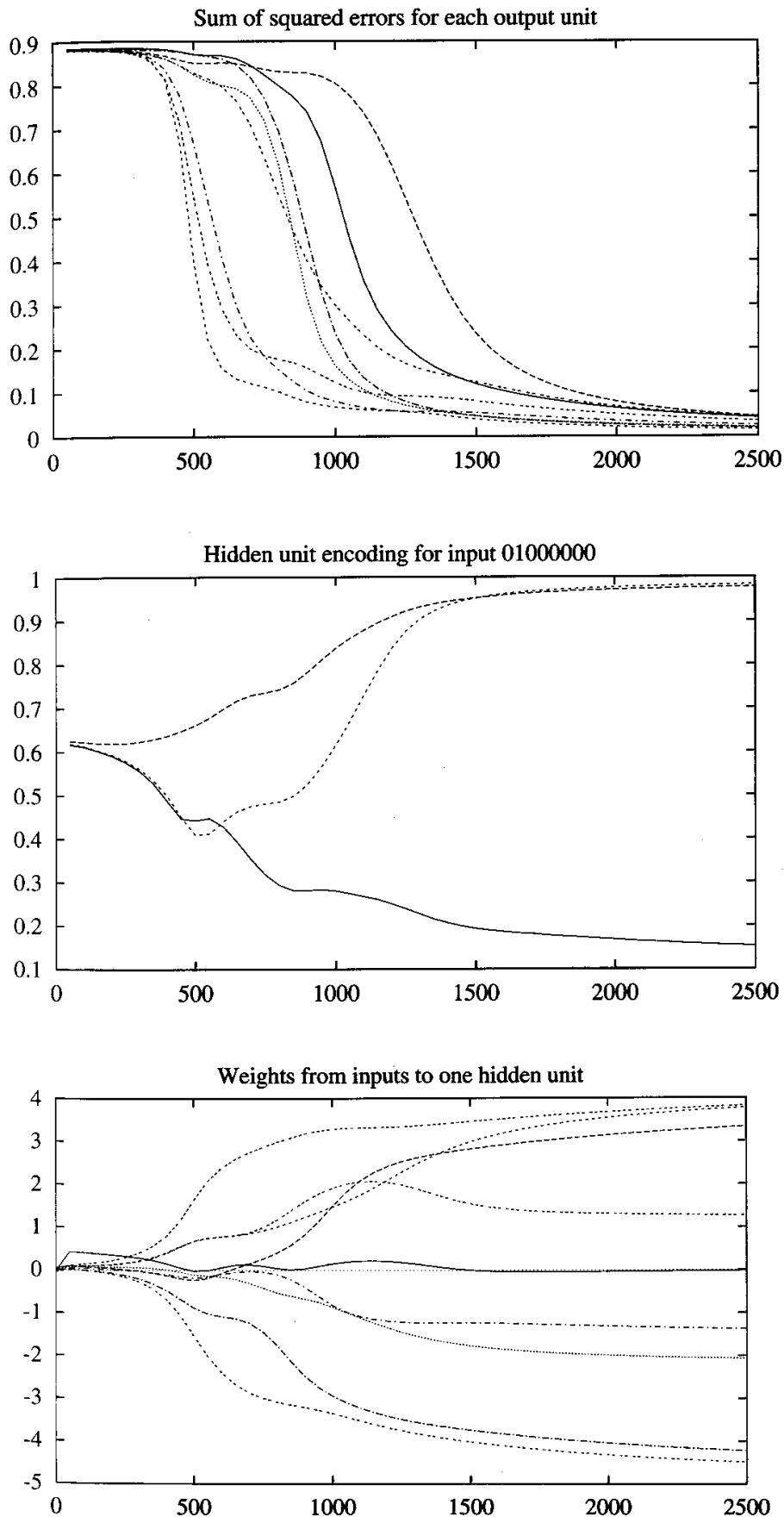
Finally, the evolution of individual weights within the network is illustrated in the third plot of Figure 4.8. This plot displays the evolution of weights connecting the eight input units (and the constant 1 bias input) to one of the three hidden units. Notice that significant changes in the weight values for this hidden unit coincide with significant changes in the hidden layer encoding and output squared errors. The weight that converges to a value near zero in this case is the bias weight w_0 .

4.6.5 Generalization, Overfitting, and Stopping Criterion

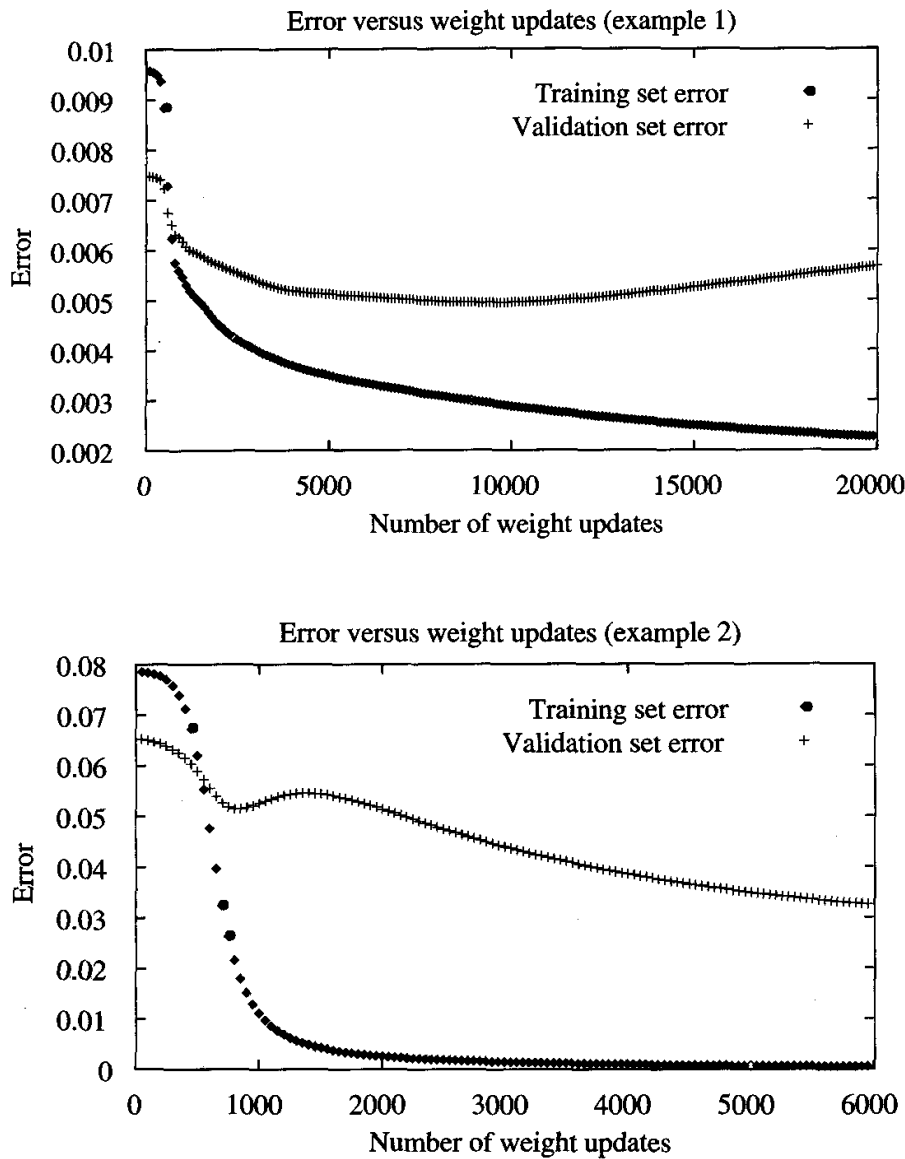
In the description of the BACKPROPAGATION algorithm in Table 4.2, the termination condition for the algorithm has been left unspecified. What is an appropriate condition for terminating the weight update loop? One obvious choice is to continue training until the error E on the training examples falls below some predetermined threshold. In fact, this is a poor strategy because BACKPROPAGATION is susceptible to overfitting the training examples at the cost of decreasing generalization accuracy over other unseen examples.

To see the dangers of minimizing the error over the training data, consider how the error E varies with the number of weight iterations. Figure 4.9 shows

[†] The source code to reproduce this example is available at <http://www.cs.cmu.edu/~tom/mlbook.html>.

**FIGURE 4.8**

Learning the $8 \times 3 \times 8$ Network. The top plot shows the evolving sum of squared errors for each of the eight output units, as the number of training iterations (epochs) increases. The middle plot shows the evolving hidden layer representation for the input string "01000000." The bottom plot shows the evolving weights for one of the three hidden units.

**FIGURE 4.9**

Plots of error E as a function of the number of weight updates, for two different robot perception tasks. In both learning cases, error E over the training examples decreases monotonically, as gradient descent minimizes this measure of error. Error over the separate “validation” set of examples typically decreases at first, then may later increase due to overfitting the training examples. The network most likely to generalize correctly to unseen data is the network with the lowest error over the validation set. Notice in the second plot, one must be careful to not stop training too soon when the validation set error begins to increase.

this variation for two fairly typical applications of BACKPROPAGATION. Consider first the top plot in this figure. The lower of the two lines shows the monotonically decreasing error E over the training set, as the number of gradient descent iterations grows. The upper line shows the error E measured over a different *validation set* of examples, distinct from the training examples. This line measures the *generalization accuracy* of the network—the accuracy with which it fits examples beyond the training data.

Notice the generalization accuracy measured over the validation examples first decreases, then increases, even as the error over the training examples continues to decrease. How can this occur? This occurs because the weights are being tuned to fit idiosyncrasies of the training examples that are not representative of the general distribution of examples. The large number of weight parameters in ANNs provides many degrees of freedom for fitting such idiosyncrasies.

Why does overfitting tend to occur during later iterations, but not during earlier iterations? Consider that network weights are initialized to small random values. With weights of nearly identical value, only very smooth decision surfaces are describable. As training proceeds, some weights begin to grow in order to reduce the error over the training data, and the complexity of the learned decision surface increases. Thus, the effective complexity of the hypotheses that can be reached by BACKPROPAGATION increases with the number of weight-tuning iterations. Given enough weight-tuning iterations, BACKPROPAGATION will often be able to create overly complex decision surfaces that fit noise in the training data or unrepresentative characteristics of the particular training sample. This overfitting problem is analogous to the overfitting problem in decision tree learning (see Chapter 3).

Several techniques are available to address the overfitting problem for BACKPROPAGATION learning. One approach, known as *weight decay*, is to decrease each weight by some small factor during each iteration. This is equivalent to modifying the definition of E to include a penalty term corresponding to the total magnitude of the network weights. The motivation for this approach is to keep weight values small, to bias learning against complex decision surfaces.

One of the most successful methods for overcoming the overfitting problem is to simply provide a set of validation data to the algorithm in addition to the training data. The algorithm monitors the error with respect to this validation set, while using the training set to drive the gradient descent search. In essence, this allows the algorithm itself to plot the two curves shown in Figure 4.9. How many weight-tuning iterations should the algorithm perform? Clearly, it should use the number of iterations that produces the lowest error *over the validation set*, since this is the best indicator of network performance over unseen examples. In typical implementations of this approach, two copies of the network weights are kept: one copy for training and a separate copy of the best-performing weights thus far, measured by their error over the validation set. Once the trained weights reach a significantly higher error over the validation set than the stored weights, training is terminated and the stored weights are returned as the final hypothesis. When this procedure is applied in the case of the top plot of Figure 4.9, it outputs the network weights obtained after 9100 iterations. The second plot in Figure 4.9 shows that it is not always obvious when the lowest error on the validation set has been reached. In this plot, the validation set error decreases, then increases, then decreases again. Care must be taken to avoid the mistaken conclusion that the network has reached its lowest validation set error at iteration 850.

In general, the issue of overfitting and how to overcome it is a subtle one. The above cross-validation approach works best when extra data are available to provide a validation set. Unfortunately, however, the problem of overfitting is most

severe for small training sets. In these cases, a k -fold cross-validation approach is sometimes used, in which cross validation is performed k different times, each time using a different partitioning of the data into training and validation sets, and the results are then averaged. In one version of this approach, the m available examples are partitioned into k disjoint subsets, each of size m/k . The cross-validation procedure is then run k times, each time using a different one of these subsets as the validation set and combining the other subsets for the training set. Thus, each example is used in the validation set for one of the experiments and in the training set for the other $k - 1$ experiments. On each experiment the above cross-validation approach is used to determine the number of iterations i that yield the best performance on the validation set. The mean $\bar{i}$ of these estimates for i is then calculated, and a final run of BACKPROPAGATION is performed *training on all n examples* for $\bar{i}$ iterations, with no validation set. This procedure is closely related to the procedure for comparing two learning methods based on limited data, described in Chapter 5.

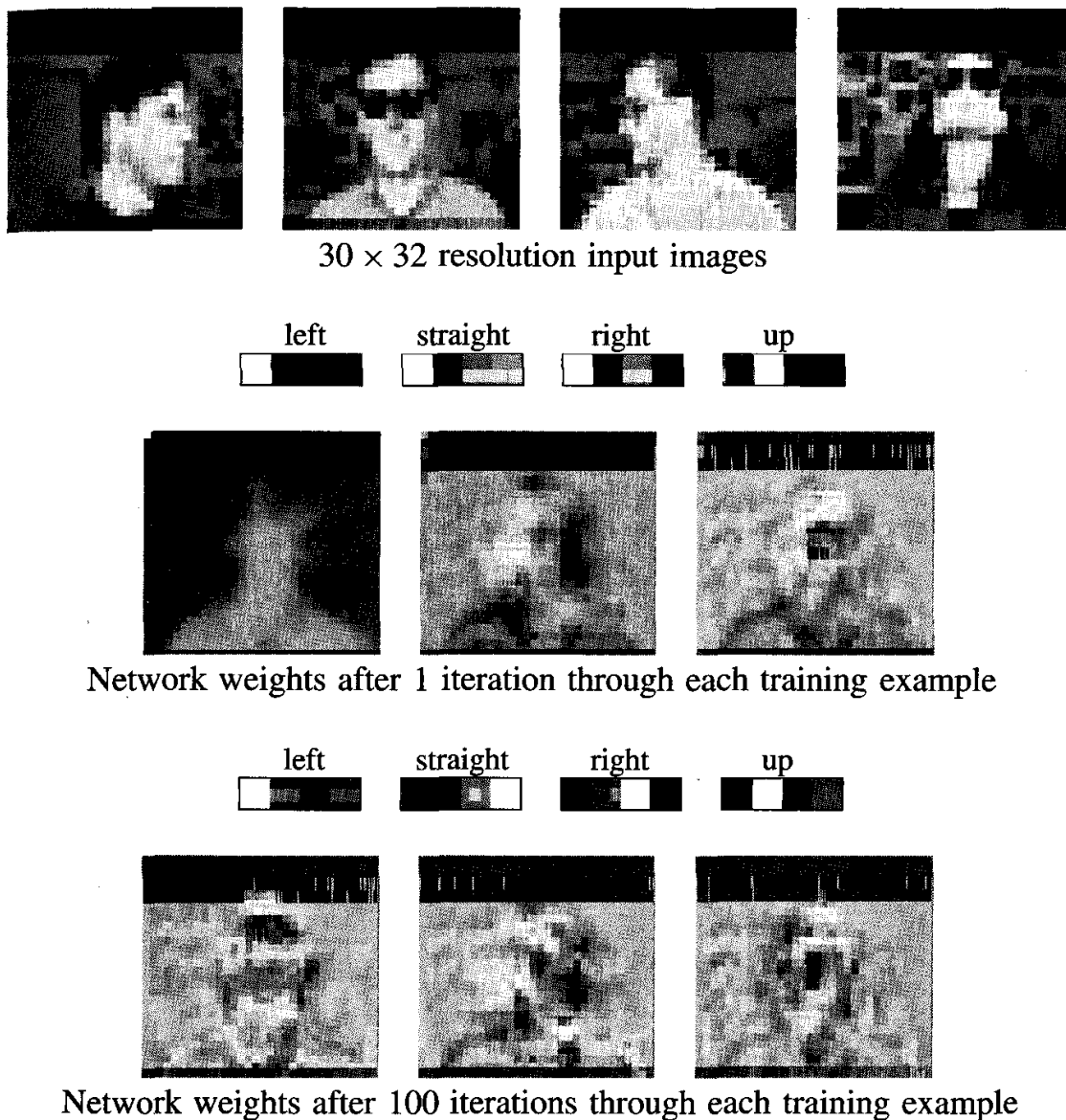
4.7 AN ILLUSTRATIVE EXAMPLE: FACE RECOGNITION

To illustrate some of the practical design choices involved in applying BACKPROPAGATION, this section discusses applying it to a learning task involving face recognition. All image data and code used to produce the examples described in this section are available at World Wide Web site <http://www.cs.cmu.edu/~tom/mlbook.html>, along with complete documentation on how to use the code. Why not try it yourself?

4.7.1 The Task

The learning task here involves classifying camera images of faces of various people in various poses. Images of 20 different people were collected, including approximately 32 images per person, varying the person's expression (happy, sad, angry, neutral), the direction in which they were looking (left, right, straight ahead, up), and whether or not they were wearing sunglasses. As can be seen from the example images in Figure 4.10, there is also variation in the background behind the person, the clothing worn by the person, and the position of the person's face within the image. In total, 624 greyscale images were collected, each with a resolution of 120×128 , with each image pixel described by a greyscale intensity value between 0 (black) and 255 (white).

A variety of target functions can be learned from this image data. For example, given an image as input we could train an ANN to output the identity of the person, the direction in which the person is facing, the gender of the person, whether or not they are wearing sunglasses, etc. All of these target functions can be learned to high accuracy from this image data, and the reader is encouraged to try out these experiments. In the remainder of this section we consider one particular task: learning the direction in which the person is facing (to their left, right, straight ahead, or upward).

**FIGURE 4.10**

Learning an artificial neural network to recognize face pose. Here a $960 \times 3 \times 4$ network is trained on grey-level images of faces (see top), to predict whether a person is looking to their left, right, ahead, or up. After training on 260 such images, the network achieves an accuracy of 90% over a separate test set. The learned network weights are shown after one weight-tuning iteration through the training examples and after 100 iterations. Each output unit (left, straight, right, up) has four weights, shown by dark (negative) and light (positive) blocks. The leftmost block corresponds to the weight w_0 , which determines the unit threshold, and the three blocks to the right correspond to weights on inputs from the three hidden units. The weights from the image pixels into each hidden unit are also shown, with each weight plotted in the position of the corresponding image pixel.

4.7.2 Design Choices

In applying BACKPROPAGATION to any given task, a number of design choices must be made. We summarize these choices below for our task of learning the direction in which a person is facing. Although no attempt was made to determine the precise optimal design choices for this task, the design described here learns

the target function quite well. After training on a set of 260 images, classification accuracy over a separate test set is 90%. In contrast, the default accuracy achieved by randomly guessing one of the four possible face directions is 25%.

Input encoding. Given that the ANN input is to be some representation of the image, one key design choice is how to encode this image. For example, we could preprocess the image to extract edges, regions of uniform intensity, or other local image features, then input these features to the network. One difficulty with this design option is that it would lead to a variable number of features (e.g., edges) per image, whereas the ANN has a fixed number of input units. The design option chosen in this case was instead to encode the image as a fixed set of 30×32 pixel intensity values, with one network input per pixel. The pixel intensity values ranging from 0 to 255 were linearly scaled to range from 0 to 1 so that network inputs would have values in the same interval as the hidden unit and output unit activations. The 30×32 pixel image is, in fact, a coarse resolution summary of the original 120×128 captured image, with each coarse pixel intensity calculated as the mean of the corresponding high-resolution pixel intensities. Using this coarse-resolution image reduces the number of inputs and network weights to a much more manageable size, thereby reducing computational demands, while maintaining sufficient resolution to correctly classify the images. Recall from Figure 4.1 that the ALVINN system uses a similar coarse-resolution image as input to the network. One interesting difference is that in ALVINN, each coarse resolution pixel intensity is obtained by selecting the intensity of a single pixel at random from the appropriate region within the high-resolution image, rather than taking the mean of all pixel intensities within this region. The motivation for this in ALVINN is that it significantly reduces the computation required to produce the coarse-resolution image from the available high-resolution image. This efficiency is especially important when the network must be used to process many images per second while autonomously driving the vehicle.

Output encoding. The ANN must output one of four values indicating the direction in which the person is looking (left, right, up, or straight). Note we could encode this four-way classification using a single output unit, assigning outputs of, say, 0.2, 0.4, 0.6, and 0.8 to encode these four possible values. Instead, we use four distinct output units, each representing one of the four possible face directions, with the highest-valued output taken as the network prediction. This is often called a *1-of- n* output encoding. There are two motivations for choosing the *1-of- n* output encoding over the single unit option. First, it provides more degrees of freedom to the network for representing the target function (i.e., there are n times as many weights available in the output layer of units). Second, in the *1-of- n* encoding the difference between the highest-valued output and the second-highest can be used as a measure of the confidence in the network prediction (ambiguous classifications may result in near or exact ties). A further design choice here is “what should be the target values for these four output units?” One obvious choice would be to use the four target values $\langle 1, 0, 0, 0 \rangle$ to encode a face looking to the

left, $\langle 0, 1, 0, 0 \rangle$ to encode a face looking straight, etc. Instead of 0 and 1 values, we use values of 0.1 and 0.9, so that $\langle 0.9, 0.1, 0.1, 0.1 \rangle$ is the target output vector for a face looking to the left. The reason for avoiding target values of 0 and 1 is that sigmoid units cannot produce these output values given finite weights. If we attempt to train the network to fit target values of exactly 0 and 1, gradient descent will force the weights to grow without bound. On the other hand, values of 0.1 and 0.9 are achievable using a sigmoid unit with finite weights.

Network graph structure. As described earlier, BACKPROPAGATION can be applied to any acyclic directed graph of sigmoid units. Therefore, another design choice we face is how many units to include in the network and how to interconnect them. The most common network structure is a layered network with feedforward connections from every unit in one layer to every unit in the next. In the current design we chose this standard structure, using two layers of sigmoid units (one hidden layer and one output layer). It is common to use one or two layers of sigmoid units and, occasionally, three layers. It is not common to use more layers than this because training times become very long and because networks with three layers of sigmoid units can already express a rich variety of target functions (see Section 4.6.2). Given our choice of a layered feedforward network with one hidden layer, how many hidden units should we include? In the results reported in Figure 4.10, only three hidden units were used, yielding a test set accuracy of 90%. In other experiments 30 hidden units were used, yielding a test set accuracy one to two percent higher. Although the generalization accuracy varied only a small amount between these two experiments, the second experiment required significantly more training time. Using 260 training images, the training time was approximately 1 hour on a Sun Sparc5 workstation for the 30 hidden unit network, compared to approximately 5 minutes for the 3 hidden unit network. In many applications it has been found that some minimum number of hidden units is required in order to learn the target function accurately and that extra hidden units above this number do not dramatically affect generalization accuracy, provided cross-validation methods are used to determine how many gradient descent iterations should be performed. If such methods are not used, then increasing the number of hidden units often increases the tendency to overfit the training data, thereby reducing generalization accuracy.

Other learning algorithm parameters. In these learning experiments the learning rate η was set to 0.3, and the momentum α was set to 0.3. Lower values for both parameters produced roughly equivalent generalization accuracy, but longer training times. If these values are set too high, training fails to converge to a network with acceptable error over the training set. Full gradient descent was used in all these experiments (in contrast to the stochastic approximation to gradient descent in the algorithm of Table 4.2). Network weights in the output units were initialized to small random values. However, input unit weights were initialized to zero, because this yields much more intelligible visualizations of the learned weights (see Figure 4.10), without any noticeable impact on generalization accuracy. The

number of training iterations was selected by partitioning the available data into a training set and a separate validation set. Gradient descent was used to minimize the error over the training set, and after every 50 gradient descent steps the performance of the network was evaluated over the validation set. The final selected network was the one with the highest accuracy over the validation set. See Section 4.6.5 for an explanation and justification of this procedure. The final reported accuracy (e.g., 90% for the network in Figure 4.10) was measured over yet a third set of test examples that were not used in any way to influence training.

4.7.3 Learned Hidden Representations

It is interesting to examine the learned weight values for the 2899 weights in the network. Figure 4.10 depicts the values of each of these weights after one iteration through the weight update for all training examples, and again after 100 iterations.

To understand this diagram, consider first the four rectangular blocks just below the face images in the figure. Each of these rectangles depicts the weights for one of the four output units in the network (encoding left, straight, right, and up). The four squares within each rectangle indicate the four weights associated with this output unit—the weight w_0 , which determines the unit threshold (on the left), followed by the three weights connecting the three hidden units to this output. The brightness of the square indicates the weight value, with bright white indicating a large positive weight, dark black indicating a large negative weight, and intermediate shades of grey indicating intermediate weight values. For example, the output unit labeled “up” has a near zero w_0 threshold weight, a large positive weight from the first hidden unit, and a large negative weight from the second hidden unit.

The weights of the hidden units are shown directly below those for the output units. Recall that each hidden unit receives an input from each of the 30×32 image pixels. The 30×32 weights associated with these inputs are displayed so that each weight is in the position of the corresponding image pixel (with the w_0 threshold weight superimposed in the top left of the array). Interestingly, one can see that the weights have taken on values that are especially sensitive to features in the region of the image in which the face and body typically appear.

The values of the network weights after 100 gradient descent iterations through each training example are shown at the bottom of the figure. Notice the leftmost hidden unit has very different weights than it had after the first iteration, and the other two hidden units have changed as well. It is possible to understand to some degree the encoding in this final set of weights. For example, consider the output unit that indicates a person is looking to his right. This unit has a strong positive weight from the second hidden unit and a strong negative weight from the third hidden unit. Examining the weights of these two hidden units, it is easy to see that if the person’s face is turned to his right (i.e., our left), then his bright skin will roughly align with strong positive weights in this hidden unit, and his dark hair will roughly align with negative weights, resulting in this unit outputting a large value. The same image will cause the third hidden unit to output a value

close to zero, as the bright face will tend to align with the large negative weights in this case.

4.8 ADVANCED TOPICS IN ARTIFICIAL NEURAL NETWORKS

4.8.1 Alternative Error Functions

As noted earlier, gradient descent can be performed for any function E that is differentiable with respect to the parameterized hypothesis space. While the basic BACKPROPAGATION algorithm defines E in terms of the sum of squared errors of the network, other definitions have been suggested in order to incorporate other constraints into the weight-tuning rule. For each new definition of E a new weight-tuning rule for gradient descent must be derived. Examples of alternative definitions of E include

- Adding a penalty term for weight magnitude. As discussed above, we can add a term to E that increases with the magnitude of the weight vector. This causes the gradient descent search to seek weight vectors with small magnitudes, thereby reducing the risk of overfitting. One way to do this is to redefine E as

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2 + \gamma \sum_{i,j} w_{ji}^2$$

which yields a weight update rule identical to the BACKPROPAGATION rule, except that each weight is multiplied by the constant $(1 - 2\gamma\eta)$ upon each iteration. Thus, choosing this definition of E is equivalent to using a weight decay strategy (see Exercise 4.10.)

- Adding a term for errors in the *slope*, or derivative of the target function. In some cases, training information may be available regarding desired derivatives of the target function, as well as desired values. For example, Simard et al. (1992) describe an application to character recognition in which certain training derivatives are used to constrain the network to learn character recognition functions that are invariant of translation within the image. Mitchell and Thrun (1993) describe methods for calculating training derivatives based on the learner's prior knowledge. In both of these systems (described in Chapter 12), the error function is modified to add a term measuring the discrepancy between these training derivatives and the actual derivatives of the learned network. One example of such an error function is

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} \left[(t_{kd} - o_{kd})^2 + \mu \sum_{j \in \text{inputs}} \left(\frac{\partial t_{kd}}{\partial x_d^j} - \frac{\partial o_{kd}}{\partial x_d^j} \right)^2 \right]$$

Here x_d^j denotes the value of the j th input unit for training example d . Thus, $\frac{\partial t_{kd}}{\partial x_d^j}$ is the training derivative describing how the target output value

t_{kd} should vary with a change in the input x_d^j . Similarly, $\frac{\partial o_{kd}}{\partial x_d^j}$ denotes the corresponding derivative of the actual learned network. The constant μ determines the relative weight placed on fitting the training values versus the training derivatives.

- Minimizing the *cross entropy* of the network with respect to the target values. Consider learning a probabilistic function, such as predicting whether a loan applicant will pay back a loan based on attributes such as the applicant's age and bank balance. Although the training examples exhibit only boolean target values (either a 1 or 0, depending on whether this applicant paid back the loan), the underlying target function might be best modeled by outputting the *probability* that the given applicant will repay the loan, rather than attempting to output the actual 1 and 0 value for each input instance. Given such situations in which we wish for the network to output probability estimates, it can be shown that the best (i.e., maximum likelihood) probability estimates are given by the network that minimizes the cross entropy, defined as

$$-\sum_{d \in D} t_d \log o_d + (1 - t_d) \log(1 - o_d)$$

Here o_d is the probability estimate output by the network for training example d , and t_d is the 1 or 0 target value for training example d . Chapter 6 discusses when and why the most probable network hypothesis is the one that minimizes this cross entropy and derives the corresponding gradient descent weight-tuning rule for sigmoid units. That chapter also describes other conditions under which the most probable hypothesis is the one that minimizes the sum of squared errors.

- Altering the effective error function can also be accomplished by weight sharing, or "tying together" weights associated with different units or inputs. The idea here is that different network weights are forced to take on identical values, usually to enforce some constraint known in advance to the human designer. For example, Waibel et al. (1989) and Lang et al. (1990) describe an application of neural networks to speech recognition, in which the network inputs are the speech frequency components at different times within a 144 millisecond time window. One assumption that can be made in this application is that the frequency components that identify a specific sound (e.g., "eee") should be independent of the exact time that the sound occurs within the 144 millisecond window. To enforce this constraint, the various units that receive input from different portions of the time window are forced to share weights. The net effect is to constrain the space of potential hypotheses, thereby reducing the risk of overfitting and improving the chances for accurately generalizing to unseen situations. Such weight sharing is typically implemented by first updating each of the shared weights separately within each unit that uses the weight, then replacing each instance of the shared weight by the mean of their values. The result of this procedure is that shared weights effectively adapt to a different error function than do the unshared weights.

4.8.2 Alternative Error Minimization Procedures

While gradient descent is one of the most general search methods for finding a hypothesis to minimize the error function, it is not always the most efficient. It is not uncommon for BACKPROPAGATION to require tens of thousands of iterations through the weight update loop when training complex networks. For this reason, a number of alternative weight optimization algorithms have been proposed and explored. To see some of the other possibilities, it is helpful to think of a weight-update method as involving two decisions: choosing a direction in which to alter the current weight vector and choosing a distance to move. In BACKPROPAGATION, the direction is chosen by taking the negative of the gradient, and the distance is determined by the learning rate constant η .

One optimization method, known as *line search*, involves a different approach to choosing the distance for the weight update. In particular, once a line is chosen that specifies the direction of the update, the update distance is chosen by finding the minimum of the error function along this line. Notice this can result in a very large or very small weight update, depending on the position of the point along the line that minimizes error. A second method, that builds on the idea of line search, is called the *conjugate gradient* method. Here, a sequence of line searches is performed to search for a minimum in the error surface. On the first step in this sequence, the direction chosen is the negative of the gradient. On each subsequent step, a new direction is chosen so that the component of the error gradient that has just been made zero, remains zero.

While alternative error-minimization methods sometimes lead to improved efficiency in training the network, methods such as conjugate gradient tend to have no significant impact on the generalization error of the final network. The only likely impact on the final error is that different error-minimization procedures may fall into different local minima. Bishop (1996) contains a general discussion of several parameter optimization methods for training networks.

4.8.3 Recurrent Networks

Up to this point we have considered only network topologies that correspond to acyclic directed graphs. Recurrent networks are artificial neural networks that apply to time series data and that use outputs of network units at time t as the input to other units at time $t + 1$. In this way, they support a form of directed cycles in the network. To illustrate, consider the time series prediction task of predicting the next day's stock market average $y(t + 1)$ based on the current day's economic indicators $x(t)$. Given a time series of such data, one obvious approach is to train a feedforward network to predict $y(t + 1)$ as its output, based on the input values $x(t)$. Such a network is shown in Figure 4.11(a).

One limitation of such a network is that the prediction of $y(t + 1)$ depends only on $x(t)$ and cannot capture possible dependencies of $y(t + 1)$ on earlier values of x . This might be necessary, for example, if tomorrow's stock market average $y(t + 1)$ depends on the difference between today's economic indicator values $x(t)$ and yesterday's values $x(t - 1)$. Of course we could remedy this difficulty

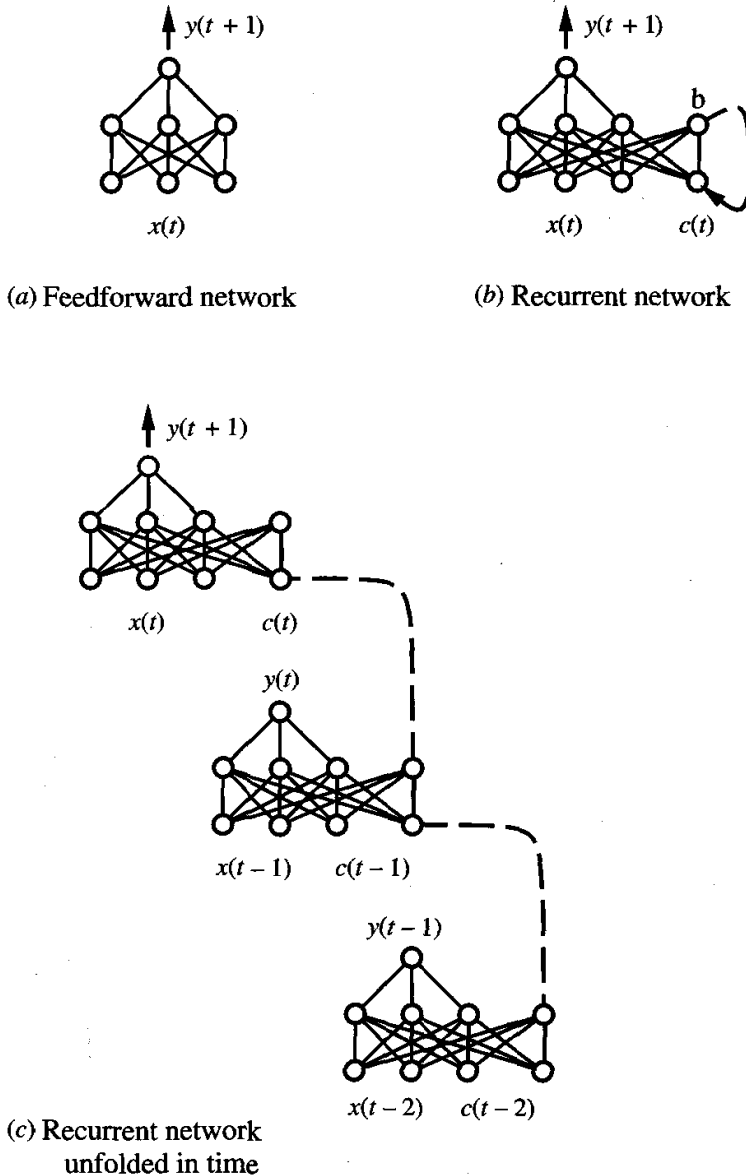


FIGURE 4.11
Recurrent networks.

by making both $x(t)$ and $x(t-1)$ inputs to the feedforward network. However, if we wish the network to consider an arbitrary window of time in the past when predicting $y(t+1)$, then a different solution is required. The recurrent network shown in Figure 4.11(b) provides one such solution. Here, we have added a new unit b to the hidden layer, and new input unit $c(t)$. The value of $c(t)$ is defined as the value of unit b at time $t-1$; that is, the input value $c(t)$ to the network at one time step is simply copied from the value of unit b on the previous time step. Notice this implements a recurrence relation, in which b represents information about the history of network inputs. Because b depends on both $x(t)$ and on $c(t)$, it is possible for b to summarize information from earlier values of x that are arbitrarily distant in time. Many other network topologies also can be used to

represent recurrence relations. For example, we could have inserted several layers of units between the input and unit b , and we could have added several context units in parallel where we added the single units b and c .

How can such recurrent networks be trained? There are several variants of recurrent networks, and several training methods have been proposed (see, for example, Jordan 1986; Elman 1990; Mozer 1995; Williams and Zipser 1995). Interestingly, recurrent networks such as the one shown in Figure 4.11(b) can be trained using a simple variant of BACKPROPAGATION. To understand how, consider Figure 4.11(c), which shows the data flow of the recurrent network “unfolded” in time. Here we have made several copies of the recurrent network, replacing the feedback loop by connections between the various copies. Notice that this large unfolded network contains no cycles. Therefore, the weights in the unfolded network can be trained directly using BACKPROPAGATION. Of course in practice we wish to keep only one copy of the recurrent network and one set of weights. Therefore, after training the unfolded network, the final weight w_{ji} in the recurrent network can be taken to be the mean value of the corresponding w_{ji} weights in the various copies. Mozer (1995) describes this training process in greater detail. In practice, recurrent networks are more difficult to train than networks with no feedback loops and do not generalize as reliably. However, they remain important due to their increased representational power.

4.8.4 Dynamically Modifying Network Structure

Up to this point we have considered neural network learning as a problem of adjusting weights within a fixed graph structure. A variety of methods have been proposed to dynamically grow or shrink the number of network units and interconnections in an attempt to improve generalization accuracy and training efficiency.

One idea is to begin with a network containing no hidden units, then grow the network as needed by adding hidden units until the training error is reduced to some acceptable level. The CASCADE-CORRELATION algorithm (Fahlman and Lebiere 1990) is one such algorithm. CASCADE-CORRELATION begins by constructing a network with no hidden units. In the case of our face-direction learning task, for example, it would construct a network containing only the four output units completely connected to the 30×32 input nodes. After this network is trained for some time, we may well find that there remains a significant residual error due to the fact that the target function cannot be perfectly represented by a network with this single-layer structure. In this case, the algorithm adds a hidden unit, choosing its weight values to maximize the correlation between the hidden unit value and the residual error of the overall network. The new unit is now installed into the network, with its weight values held fixed, and a new connection from this new unit is added to each output unit. The process is now repeated. The original weights are retrained (holding the hidden unit weights fixed), the residual error is checked, and a second hidden unit added if the residual error is still above threshold. Whenever a new hidden unit is added, its inputs include all of the original network inputs plus the outputs of any existing hidden units. The network is

grown in this fashion, accumulating hidden units until the network residual error is reduced to some acceptable level. Fahlman and Lebiere (1990) report cases in which CASCADE-CORRELATION significantly reduces training times, due to the fact that only a single layer of units is trained at each step. One practical difficulty is that because the algorithm can add units indefinitely, it is quite easy for it to overfit the training data, and precautions to avoid overfitting must be taken.

A second idea for dynamically altering network structure is to take the opposite approach. Instead of beginning with the simplest possible network and adding complexity, we begin with a complex network and prune it as we find that certain connections are inessential. One way to decide whether a particular weight is inessential is to see whether its value is close to zero. A second way, which appears to be more successful in practice, is to consider the effect that a small variation in the weight has on the error E . The effect on E of varying w (i.e., $\frac{\partial E}{\partial w}$) can be taken as a measure of the salience of the connection. LeCun et al. (1990) describe a process in which a network is trained, the least salient connections removed, and this process iterated until some termination condition is met. They refer to this as the “optimal brain damage” approach, because at each step the algorithm attempts to remove the least useful connections. They report that in a character recognition application this approach reduced the number of weights in a large network by a factor of 4, with a slight improvement in generalization accuracy and a significant improvement in subsequent training efficiency.

In general, techniques for dynamically modifying network structure have met with mixed success. It remains to be seen whether they can reliably improve on the generalization accuracy of BACKPROPAGATION. However, they have been shown in some cases to provide significant improvements in training times.

4.9 SUMMARY AND FURTHER READING

Main points of this chapter include:

- Artificial neural network learning provides a practical method for learning real-valued and vector-valued functions over continuous and discrete-valued attributes, in a way that is robust to noise in the training data. The BACKPROPAGATION algorithm is the most common network learning method and has been successfully applied to a variety of learning tasks, such as handwriting recognition and robot control.
- The hypothesis space considered by the BACKPROPAGATION algorithm is the space of all functions that can be represented by assigning weights to the given, fixed network of interconnected units. Feedforward networks containing three layers of units are able to approximate *any* function to arbitrary accuracy, given a sufficient (potentially very large) number of units in each layer. Even networks of practical size are capable of representing a rich space of highly nonlinear functions, making feedforward networks a good choice for learning discrete and continuous functions whose general form is unknown in advance.

- **BACKPROPAGATION** searches the space of possible hypotheses using gradient descent to iteratively reduce the error in the network fit to the training examples. Gradient descent converges to a local minimum in the training error with respect to the network weights. More generally, gradient descent is a potentially useful method for searching many continuously parameterized hypothesis spaces where the training error is a differentiable function of hypothesis parameters.
- One of the most intriguing properties of **BACKPROPAGATION** is its ability to invent new features that are not explicit in the input to the network. In particular, the internal (hidden) layers of multilayer networks learn to represent intermediate features that are useful for learning the target function and that are only implicit in the network inputs. This capability is illustrated, for example, by the ability of the $8 \times 3 \times 8$ network in Section 4.6.4 to invent the boolean encoding of digits from 1 to 8 and by the image features represented by the hidden layer in the face-recognition application of Section 4.7.
- Overfitting the training data is an important issue in ANN learning. Overfitting results in networks that generalize poorly to new data despite excellent performance over the training data. Cross-validation methods can be used to estimate an appropriate stopping point for gradient descent search and thus to minimize the risk of overfitting.
- Although **BACKPROPAGATION** is the most common ANN learning algorithm, many others have been proposed, including algorithms for more specialized tasks. For example, recurrent neural network methods train networks containing directed cycles, and algorithms such as **CASCADE CORRELATION** alter the network structure as well as the network weights.

Additional information on ANN learning can be found in several other chapters in this book. A Bayesian justification for choosing to minimize the sum of squared errors is given in Chapter 6, along with a justification for minimizing the cross-entropy instead of the sum of squared errors in other cases. Theoretical results characterizing the number of training examples needed to reliably learn boolean functions and the Vapnik-Chervonenkis dimension of certain types of networks can be found in Chapter 7. A discussion of overfitting and how to avoid it can be found in Chapter 5. Methods for using prior knowledge to improve the generalization accuracy of ANN learning are discussed in Chapter 12.

Work on artificial neural networks dates back to the very early days of computer science. McCulloch and Pitts (1943) proposed a model of a neuron that corresponds to the perceptron, and a good deal of work through the 1960s explored variations of this model. During the early 1960s Widrow and Hoff (1960) explored perceptron networks (which they called “adelines”) and the delta rule, and Rosenblatt (1962) proved the convergence of the perceptron training rule. However, by the late 1960s it became clear that single-layer perceptron networks had limited representational capabilities, and no effective algorithms were known for training multilayer networks. Minsky and Papert (1969) showed that even

simple functions such as XOR could not be represented or learned with single-layer perceptron networks, and work on ANNs receded during the 1970s.

During the mid-1980s work on ANNs experienced a resurgence, caused in large part by the invention of BACKPROPAGATION and related algorithms for training multilayer networks (Rumelhart and McClelland 1986; Parker 1985). These ideas can be traced to related earlier work (e.g., Werbos 1975). Since the 1980s, BACKPROPAGATION has become a widely used learning method, and many other ANN approaches have been actively explored. The advent of inexpensive computers during this same period has allowed experimenting with computationally intensive algorithms that could not be thoroughly explored during the 1960s.

A number of textbooks are devoted to the topic of neural network learning. An early but still useful book on parameter learning methods for pattern recognition is Duda and Hart (1973). The text by Widrow and Stearns (1985) covers perceptrons and related single-layer networks and their applications. Rumelhart and McClelland (1986) produced an edited collection of papers that helped generate the increased interest in these methods beginning in the mid-1980s. Recent books on neural network learning include Bishop (1996); Chauvin and Rumelhart (1995); Freeman and Skapina (1991); Fu (1994); Hecht-Nielsen (1990); and Hertz et al. (1991).

EXERCISES

- 4.1. What are the values of weights w_0 , w_1 , and w_2 for the perceptron whose decision surface is illustrated in Figure 4.3? Assume the surface crosses the x_1 axis at -1 , and the x_2 axis at 2 .
- 4.2. Design a two-input perceptron that implements the boolean function $A \wedge \neg B$. Design a two-layer network of perceptrons that implements $A \text{ XOR } B$.
- 4.3. Consider two perceptrons defined by the threshold expression $w_0 + w_1x_1 + w_2x_2 > 0$. Perceptron A has weight values

$$w_0 = 1, \quad w_1 = 2, \quad w_2 = 1$$

and perceptron B has the weight values

$$w_0 = 0, \quad w_1 = 2, \quad w_2 = 1$$

True or false? Perceptron A is *more general than* perceptron B. (*more general than* is defined in Chapter 2.)

- 4.4. Implement the delta training rule for a two-input linear unit. Train it to fit the target concept $-2 + x_1 + 2x_2 > 0$. Plot the error E as a function of the number of training iterations. Plot the decision surface after 5, 10, 50, 100, $\dots$, iterations.
 - (a) Try this using various constant values for η and using a decaying learning rate of η_0/i for the i th iteration. Which works better?
 - (b) Try incremental and batch learning. Which converges more quickly? Consider both number of weight updates and total execution time.
- 4.5. Derive a gradient descent training rule for a single unit with output o , where

$$o = w_0 + w_1x_1 + w_1x_1^2 + \dots + w_nx_n + w_nx_n^2$$

- 4.6. Explain informally why the delta training rule in Equation (4.10) is only an approximation to the true gradient descent rule of Equation (4.7).
- 4.7. Consider a two-layer feedforward ANN with two inputs a and b , one hidden unit c , and one output unit d . This network has five weights ($w_{ca}, w_{cb}, w_{c0}, w_{dc}, w_{d0}$), where w_{x0} represents the threshold weight for unit x . Initialize these weights to the values $(.1, .1, .1, .1, .1)$, then give their values after each of the first two training iterations of the BACKPROPAGATION algorithm. Assume learning rate $\eta = .3$, momentum $\alpha = 0.9$, incremental weight updates, and the following training examples:

a	b	d
1	0	1
0	1	0

- 4.8. Revise the BACKPROPAGATION algorithm in Table 4.2 so that it operates on units using the squashing function $\tanh$ in place of the sigmoid function. That is, assume the output of a single unit is $o = \tanh(\vec{w} \cdot \vec{x})$. Give the weight update rule for output layer weights and hidden layer weights. Hint: $\tanh'(x) = 1 - \tanh^2(x)$.
- 4.9. Recall the $8 \times 3 \times 8$ network described in Figure 4.7. Consider trying to train a $8 \times 1 \times 8$ network for the same task; that is, a network with just one hidden unit. Notice the eight training examples in Figure 4.7 could be represented by eight distinct values for the single hidden unit (e.g., 0.1, 0.2, ..., 0.8). Could a network with just one hidden unit therefore learn the identity function defined over these training examples? Hint: Consider questions such as “do there exist values for the hidden unit weights that can create the hidden unit encoding suggested above?” “do there exist values for the output unit weights that could correctly decode this encoding of the input?” and “is gradient descent likely to find such weights?”
- 4.10. Consider the alternative error function described in Section 4.8.1

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2 + \gamma \sum_{i,j} w_{ji}^2$$

Derive the gradient descent update rule for this definition of E . Show that it can be implemented by multiplying each weight by some constant before performing the standard gradient descent update given in Table 4.2.

- 4.11. Apply BACKPROPAGATION to the task of face recognition. See World Wide Web URL <http://www.cs.cmu.edu/~tom/book.html> for details, including face-image data, BACKPROPAGATION code, and specific tasks.
- 4.12. Consider deriving a gradient descent algorithm to learn target concepts corresponding to rectangles in the x, y plane. Describe each hypothesis by the x and y coordinates of the lower-left and upper-right corners of the rectangle — llx, lly, urx , and ury respectively. An instance $\langle x, y \rangle$ is labeled positive by hypothesis $\langle llx, lly, urx, ury \rangle$ if and only if the point $\langle x, y \rangle$ lies inside the corresponding rectangle. Define error E as in the chapter. Can you devise a gradient descent algorithm to learn such rectangle hypotheses? Notice that E is not a continuous function of llx, lly, urx , and ury , just as in the case of perceptron learning. (Hint: Consider the two solutions used for perceptrons: (1) changing the classification rule to make output predictions *continuous* functions of the inputs, and (2) defining an alternative error—such as distance to the rectangle center—as in using the delta rule to train perceptrons.) Does your algorithm converge to the minimum error hypothesis when the positive and negative examples are separable by a rectangle? When they are not? Do you

have problems with local minima? How does your algorithm compare to symbolic methods for learning conjunctions of feature constraints?

REFERENCES

- Bishop, C. M. (1996). *Neural networks for pattern recognition*. Oxford, England: Oxford University Press.
- Chauvin, Y., & Rumelhart, D. (1995). *BACKPROPAGATION: Theory, architectures, and applications* (edited collection). Hillsdale, NJ: Lawrence Erlbaum Assoc.
- Churchland, P. S., & Sejnowski, T. J. (1992). *The computational brain*. Cambridge, MA: The MIT Press.
- Cybenko, G. (1988). Continuous valued neural networks with two hidden layers are sufficient (Technical Report). Department of Computer Science, Tufts University, Medford, MA.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2, 303–314.
- Cottrell, G. W. (1990). Extracting features from faces using compression networks: Face, identity, emotion and gender recognition using holons. In D. Touretzky (Ed.), *Connection Models: Proceedings of the 1990 Summer School*. San Mateo, CA: Morgan Kaufmann.
- Dietterich, T. G., Hild, H., & Bakiri, G. (1995). A comparison of ID3 and BACKPROPAGATION for English text-to-speech mapping. *Machine Learning*, 18(1), 51–80.
- Duda, R., & Hart, P. (1973). *Pattern classification and scene analysis*. New York: John Wiley & Sons.
- Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14, 179–211.
- Fahlman, S., & Lebiere, C. (1990). *The CASCADE-CORRELATION learning architecture* (Technical Report CMU-CS-90-100). Computer Science Department, Carnegie Mellon University, Pittsburgh, PA.
- Freeman, J. A., & Skapura, D. M. (1991). *Neural networks*. Reading, MA: Addison Wesley.
- Fu, L. (1994). *Neural networks in computer intelligence*. New York: McGraw Hill.
- Gabriel, M., & Moore, J. (1990). *Learning and computational neuroscience: Foundations of adaptive networks* (edited collection). Cambridge, MA: The MIT Press.
- Hecht-Nielsen, R. (1990). *Neurocomputing*. Reading, MA: Addison Wesley.
- Hertz, J., Krogh, A., & Palmer, R. G. (1991). *Introduction to the theory of neural computation*. Reading, MA: Addison Wesley.
- Hornick, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2, 359–366.
- Huang, W. Y., & Lippmann, R. P. (1988). Neural net and traditional classifiers. In Anderson (Ed.), *Neural Information Processing Systems* (pp. 387–396).
- Jordan, M. (1986). Attractor dynamics and parallelism in a connectionist sequential machine. *Proceedings of the Eighth Annual Conference of the Cognitive Science Society* (pp. 531–546).
- Kohonen, T. (1984). *Self-organization and associative memory*. Berlin: Springer-Verlag.
- Lang, K. J., Waibel, A. H., & Hinton, G. E. (1990). A time-delay neural network architecture for isolated word recognition. *Neural Networks*, 3, 33–43.
- LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). BACKPROPAGATION applied to handwritten zip code recognition. *Neural Computation*, 1(4).
- LeCun, Y., Denker, J. S., & Solla, S. A. (1990). Optimal brain damage. In D. Touretzky (Ed.), *Advances in Neural Information Processing Systems* (Vol. 2, pp. 598–605). San Mateo, CA: Morgan Kaufmann.
- Manke, S., Finke, M., & Waibel, A. (1995). NPEN++: a writer independent, large vocabulary online cursive handwriting recognition system. *Proceedings of the International Conference on Document Analysis and Recognition*. Montreal, Canada: IEEE Computer Society.
- McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5, 115–133.

- Mitchell, T. M., & Thrun, S. B. (1993). Explanation-based neural network learning for robot control. In Hanson, Cowan, & Giles (Eds.), *Advances in neural information processing systems 5* (pp. 287–294). San Francisco: Morgan Kaufmann.
- Mozer, M. (1995). A focused BACKPROPAGATION algorithm for temporal pattern recognition. In Y. Chauvin & D. Rumelhart (Eds.), *Backpropagation: Theory, architectures, and applications* (pp. 137–169). Hillsdale, NJ: Lawrence Erlbaum Associates.
- Minsky, M., & Papert, S. (1969). *Perceptrons*. Cambridge, MA: MIT Press.
- Nilsson, N. J. (1965). *Learning machines*. New York: McGraw Hill.
- Parker, D. (1985). *Learning logic* (MIT Technical Report TR-47). MIT Center for Research in Computational Economics and Management Science.
- Pomerleau, D. A. (1993). Knowledge-based training of artificial neural networks for autonomous robot driving. In J. Connell & S. Mahadevan (Eds.), *Robot Learning* (pp. 19–43). Boston: Kluwer Academic Publishers.
- Rosenblatt, F. (1959). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65, 386–408.
- Rosenblatt, F. (1962). *Principles of neurodynamics*. New York: Spartan Books.
- Rumelhart, D. E., & McClelland, J. L. (1986). *Parallel distributed processing: exploration in the microstructure of cognition* (Vols. 1 & 2). Cambridge, MA: MIT Press.
- Rumelhart, D., Widrow, B., & Lehr, M. (1994). The basic ideas in neural networks. *Communications of the ACM*, 37(3), 87–92.
- Shavlik, J. W., Mooney, R. J., & Towell, G. G. (1991). Symbolic and neural learning algorithms: An experimental comparison. *Machine Learning*, 6(2), 111–144.
- Simard, P. S., Victorri, B., LeCun, Y., & Denker, J. (1992). Tangent prop—A formalism for specifying selected invariances in an adaptive network. In Moody, et al. (Eds.), *Advances in Neural Information Processing Systems 4* (pp. 895–903). San Francisco: Morgan Kaufmann.
- Waibel, A., Hanazawa, T., Hinton, G., Shikano, K., & Lang, K. (1989). Phoneme recognition using time-delay neural networks. *IEEE Transactions on Acoustics, Speech and Signal Processing*.
- Weiss, S., & Kapouleas, I. (1989). An empirical comparison of pattern recognition, neural nets, and machine learning classification methods. *Proceedings of the Eleventh IJCAI* (pp. 781–787). San Francisco: Morgan Kaufmann.
- Werbos, P. (1975). Beyond regression: *New tools for prediction and analysis in the behavioral sciences* (Ph.D. dissertation). Harvard University.
- Widrow, B., & Hoff, M. E. (1960). Adaptive switching circuits. *IRE WESCON Convention Record*, 4, 96–104.
- Widrow, B., & Stearns, S. D. (1985). *Adaptive signal processing*. Signal Processing Series. Englewood Cliffs, NJ: Prentice Hall.
- Williams, R., & Zipser, D. (1995). Gradient-based learning algorithms for recurrent networks and their computational complexity. In Y. Chauvin & D. Rumelhart (Eds.), *Backpropagation: Theory, architectures, and applications* (pp. 433–486). Hillsdale, NJ: Lawrence Erlbaum Associates.
- Zornetzer, S. F., Davis, J. L., & Lau, C. (1994). *An introduction to neural and electronic networks* (edited collection) (2nd ed.). New York: Academic Press.

CHAPTER 5

EVALUATING HYPOTHESES

Empirically evaluating the accuracy of hypotheses is fundamental to machine learning. This chapter presents an introduction to statistical methods for estimating hypothesis accuracy, focusing on three questions. First, given the observed accuracy of a hypothesis over a limited sample of data, how well does this estimate its accuracy over additional examples? Second, given that one hypothesis outperforms another over some sample of data, how probable is it that this hypothesis is more accurate in general? Third, when data is limited what is the best way to use this data to both learn a hypothesis and estimate its accuracy? Because limited samples of data might misrepresent the general distribution of data, estimating true accuracy from such samples can be misleading. Statistical methods, together with assumptions about the underlying distributions of data, allow one to bound the difference between observed accuracy over the sample of available data and the true accuracy over the entire distribution of data.

5.1 MOTIVATION

In many cases it is important to evaluate the performance of learned hypotheses as precisely as possible. One reason is simply to understand whether to use the hypothesis. For instance, when learning from a limited-size database indicating the effectiveness of different medical treatments, it is important to understand as precisely as possible the accuracy of the learned hypotheses. A second reason is that evaluating hypotheses is an integral component of many learning methods. For example, in post-pruning decision trees to avoid overfitting, we must evaluate